

Build School 課程 – C# 基礎課程 07

Bill Chung
V2026.1 台北夏季班

請尊重講師的著作權及智慧財產權!

Build School 課程之教材、程式碼等、僅供課程中學習用、請不要任意自行散佈、重製、分享，謝謝

DoWhere 總整理

LAB

DoWhere(1) 委派

建立新的方案及專案

- 方案名稱：DoWhereSamples
- 專案名稱：DoWhereSample001
- 範本：Console Application
- 同 CH5\DelegateSample005

建立 MyClass

```
public class MyClass
{
    public static List<string> DoWhere(List<string> source,
                                       Func<string, bool> predicate)
    {
        List<string> result = new List<string>();
        foreach (var item in source)
        {
            if (predicate.Invoke(item))
            {
                result.Add(item);
            }
        }
        return result;
    }
}
```

修改 Main method

```
static void Main(string[] args)
{
    // 使用集合運算式 (Collection Expression) 建立 List<string>
    List<string> source = [ "Bill", "John", "David", "Tom", "David" ];
    var result = MyClass.DoWhere(source, x => x.Length == 4);
    Console.WriteLine(string.Join(",", result));
    Console.ReadLine();
}
```

執行

LAB

DoWhere(2) 擴充方法

建立新的方案及專案

- 方案名稱：DoWhereSamples
- 專案名稱：DoWhereSample002
- 範本：Console Application

複製範例一的 MyClass 後修改為擴充方法形式

```
public static class MyClass
{
    public static List<string> DoWhere(this List<string> source,
                                       Func<string, bool> predicate)
    {
        List<string> result = new List<string>();
        foreach (var item in source)
        {
            if (predicate.Invoke(item))
            {
                result.Add(item);
            }
        }
        return result;
    }
}
```

修改 Main

```
static void Main(string[] args)
{
    List<string> source = ["Bill", "John", "David", "Tom", "David"];
    // 改使用擴充方法語法
    var result = source.DoWhere(x => x.Length == 4);
    /* 等同於
     * var result = MyClass.DoWhere(source, x => x.Length == 4);
     */
    Console.WriteLine(string.Join(",", result));
    Console.ReadLine();
}
```

執行

LAB

DoWhere(3) 泛型

加入新專案

- 方案名稱：DoWhereSamples
- 專案名稱：DoWhereSample003
- 範本：Console Application

複製範例二的 MyClass 後修改為泛型擴充方法形式

(1) 先修改 DoWhere 為 DoWhere<T>

(2) 接著再將所有的 string 宣告改為 T

```
public static class MyClass
{
    public static List<T> DoWhere<T>(this List<T> source,
                                     Func<T, bool> predicate)
    {
        List<T> result = new List<T>();
        foreach (var item in source)
        {
            if (predicate.Invoke(item))
            {
                result.Add(item);
            }
        }
        return result;
    }
}
```


修改 Main

現在不只 List<string> 可以用了

```
static void Main(string[] args)
{
    List<string> source = ["Bill", "John", "David", "Tom", "David"];

    var result = source.DoWhere(x => x.Length == 4);
    Console.WriteLine(string.Join(", ", result));

    {
        List<int> source2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
        var result2 = source2.DoWhere(x => x % 2 == 0);
        Console.WriteLine(string.Join(", ", result2));
        Console.ReadLine();
    }
}
```

執行

LAB

DoWhere(4) IEnumerable<T>

加入新專案

- 方案名稱：DoWhereSamples
- 專案名稱：DoWhereSample004
- 範本：Console Application

說明

- 因為 DoWhere 內部對 source 的操作只有 foreach，而 IEnumerable<T> 正好是宣告自己可以被 foreach
- 回傳 IEnumerable<T> 的原因
 - 呼叫端可能只需要 foreach
 - 如果呼叫端需要其他形式，呼叫端可以自行決定
 - 為下一節的迭代器模式做準備

複製範例三的 MyClass

(1) 將 DoWhere 參數型別改為 IEnumerable<T>

(2) 將 DoWhere 回傳值型別改為 IEnumerable<T>

```
public static class MyClass
{
    public static IEnumerable<T> DoWhere<T>(this IEnumerable<T> source,
                                             Func<T, bool> predicate)
    {
        List<T> result = new List<T>();
        foreach (var item in source)
        {
            if (predicate.Invoke(item))
            {
                result.Add(item);
            }
        }
        return result;
    }
}
```

修改 Main，將 List<int> 改為 int[]

```
static void Main(string[] args)
{
    List<string> source = ["Bill", "John", "David", "Tom", "David"];
    var result = source.DoWhere(x => x.Length == 4);
    Console.WriteLine(string.Join(", ", result));

    int[] source2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
    var result2 = source2.DoWhere(x => x % 2 == 0);
    Console.WriteLine(string.Join(", ", result2));
    Console.ReadLine();
}
```



執行

迭代器

迭代器 Iterator

- 在 C# 又稱為列舉器 (Enumerator)
- 提供一種方式能夠依序存取物件中的每一個元素，但不需要知道聚合物件 (aggregate object) 的內部細節。
- 實作或回傳 IEnumerable 或 IEnumerable<T>

yield

- 在陳述式中使用 `yield` 關鍵字時，表示關鍵字所在的方法、運算子或 `get` 存取子是迭代器。
- `yield return <expression>`
- `yield break`

LAB

使用 yield return

建立新的方案及專案

- 方案名稱：IteratorSamples
- 專案名稱：IteratorSample001
- 範本：Console Application

修改 Main

```
static void Main(string[] args)
{
    foreach (var item in Exec())
    {
        Console.WriteLine(item);
    }
    Console.ReadLine();
}

static IEnumerable<string> Exec()
{
    yield return "A";
    yield return "B";
    yield return "C";
}
```



執行

LAB

foreach 的秘密

跟著老師做

- 一步步修改 `IteratorSample001`

LAB

DoWhere(5) yield return

加入新專案

- 方案名稱：DoWhereSamples
- 專案名稱：DoWhereSample005
- 範本：Console Application

複製範例四的 MyClass

(1) 移除 `List<T> result = new List<T>();`

(2) 移除 `return result;`

(3) 將 `result.Add(item)` 改為 `yield return`

```
public static class MyClass
{
    public static IEnumerable<T> DoWhere<T>(this IEnumerable<T> source,
                                             Func<T, bool> predicate)
    {
        foreach (var item in source)
        {
            if (predicate.Invoke(item))
            {
                yield return item;
            }
        }
    }
}
```

Program class 與 004 相同

```
static void Main(string[] args)
{
    List<string> source = ["Bill", "John", "David", "Tom", "David"];
    var result = source.DoWhere(x => x.Length == 4);
    Console.WriteLine(string.Join(", ", result));

    int[] source2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
    var result2 = source2.DoWhere(x => x % 2 == 0);
    Console.WriteLine(string.Join(", ", result2));
    Console.ReadLine();
}
```

討論

- 回顧 DoWhere 1~5，
討論你們在每個階段學
到甚麼樣的知識



LAB

DoWhere(6)

用 delegate 和 interface/class 的差別

加入新專案

- 方案名稱：DoWhereSamples
- 專案名稱：DoWhereSample006
- 範本：Console Application

建立 interface 來替代 Func<T, bool>

```
public interface IPredicate<T>
{
    bool Invoke(T item);
}
```

複製範例五的 MyClass

將第二個參數的型別由 Func<T,bool> 改為 IPredicate<T>

```
public static class MyClass
{
    public static IEnumerable<T> DoWhere<T>(this IEnumerable<T> source,
                                             IPredicte<T> predicate)
    {
        foreach (var item in source)
        {
            if (predicate.Invoke(item))
            {
                yield return item;
            }
        }
    }
}
```

為兩個比較建立個別的 class
兩個類別都要實作 IPredicate<T> 介面

```
public class EvenPredicate : IPredicate<int>
{
    public bool Invoke(int item)
    {
        return item % 2 == 0;
    }
}

public class Length4Predicate : IPredicate<string>
{
    public bool Invoke(string item)
    {
        return item.Length == 4;
    }
}
```

複製 Sample005 的 Program class 修改兩個 DoWhere 的引數

```
static void Main(string[] args)
{
    List<string> source = ["Bill", "John", "David", "Tom", "David"];
    var result = source.DoWhere(new Length4Predicate());
    Console.WriteLine(string.Join(",", result));

    int[] source2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
    var result2 = source2.DoWhere(new EvenPredicate());
    Console.WriteLine(string.Join(",", result2));
    Console.ReadLine();
}
```

執行

討論

- 能否正確地解釋委派的意義，以及相對於使用介面/類別的差異。



隨堂練習

自己寫 Skip 與 Take

- 說明
 - 開啟範例的 `CH7\CustomSkipTakeExercise.slnx`
 - 完成 `MySkip` 與 `MyTake` 的內容

Entity Framework

C# 的資料庫處理

- 目前我們使用 C# 處理資料庫的方式大概有幾種
 - 傳統的 ADO.NET
 - ADO.NET Entity Framework
 - 第三方的 ORM Framework
- 很重要的文件：ADO.NET
 - [https://msdn.microsoft.com/zh-tw/library/e80y5yhx\(v=vs.110\).aspx](https://msdn.microsoft.com/zh-tw/library/e80y5yhx(v=vs.110).aspx)

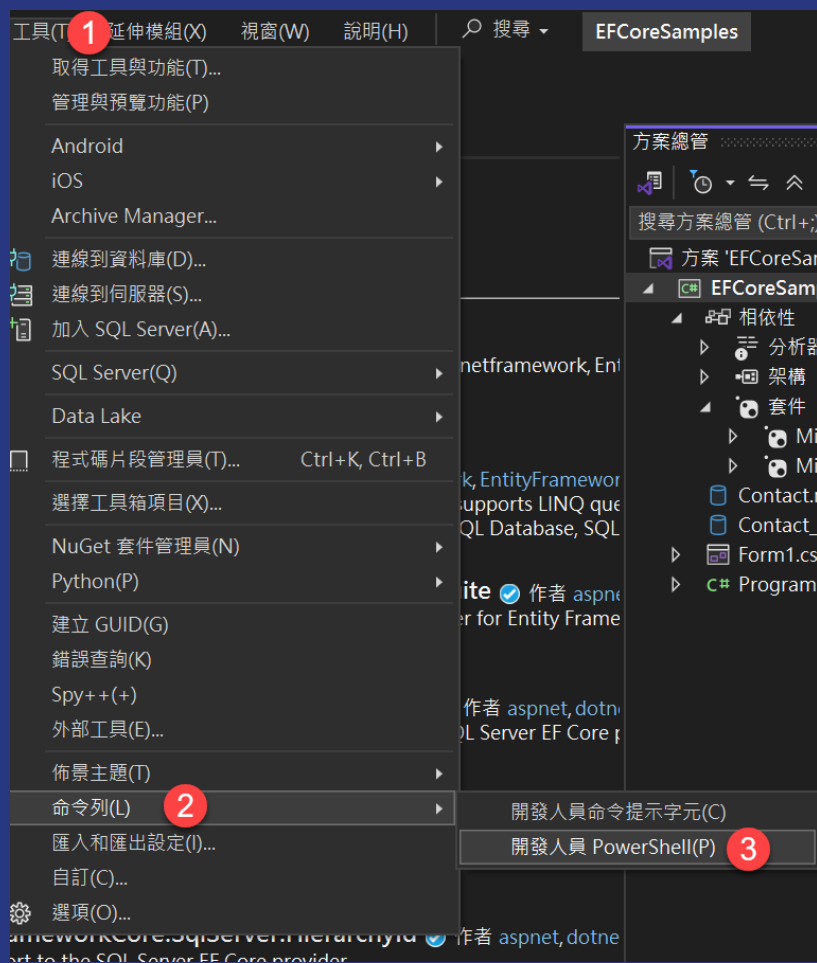
資料模型開發策略

- Database First
- Code First

Entity Framework Core 工具

- [.NET CLI EF Core tool](#)
- [EF Core Power Tools](#)
- [其他](#)

如何開啟 powershell



local db 命令 (powershell)

- 資訊 *sqllocaldb info mssqllocaldb*
- 啟動 *sqllocaldb start mssqllocaldb*
- 停止 *sqllocaldb stop mssqllocaldb*
- 強制停止 *sqllocaldb stop mssqllocaldb -k*
- 刪除 *sqllocaldb delete mssqllocaldb*
- 重建 *sqllocaldb create mssqllocaldb*

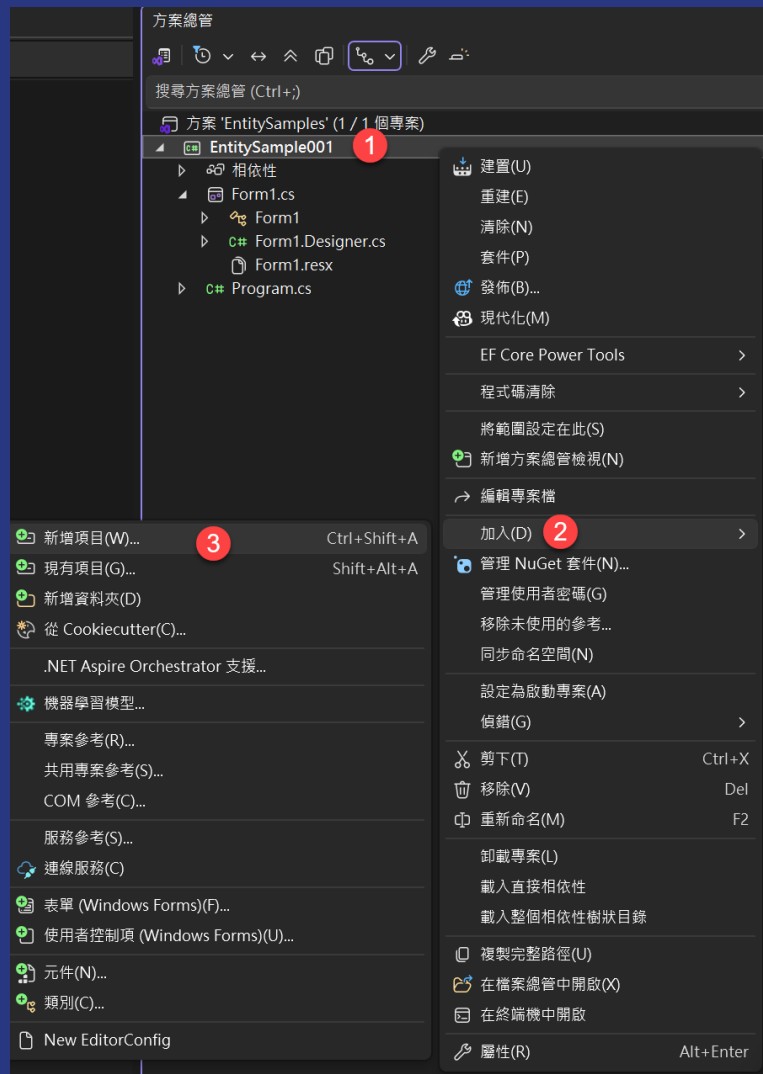
LAB

建立 Local DB

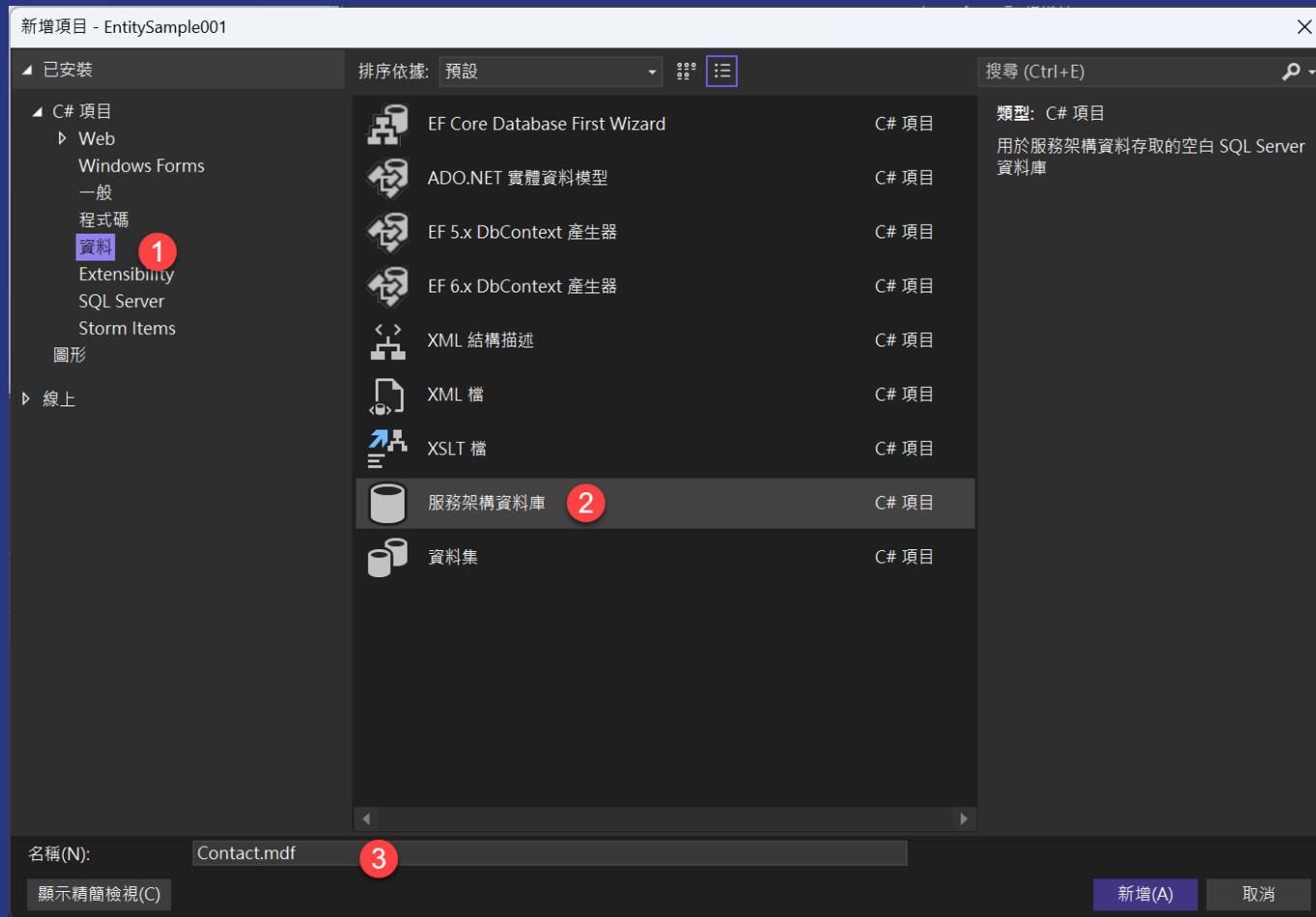
新增一個方案及專案

- 方案名稱：EntitySamples
- 專案名稱：EntitySample001
- 範本：Windows Forms Application

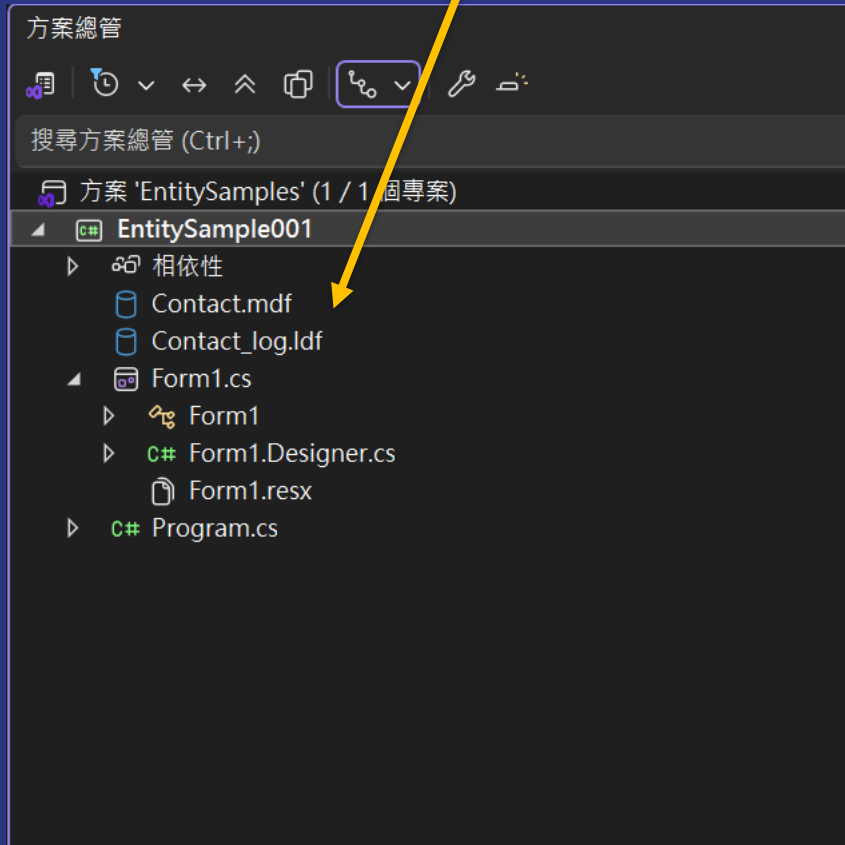
首先來加入一個 local db



項目範本選【資料】→【服務架構資料庫】，
並命名為 Contact.mdf
然後按【新增】



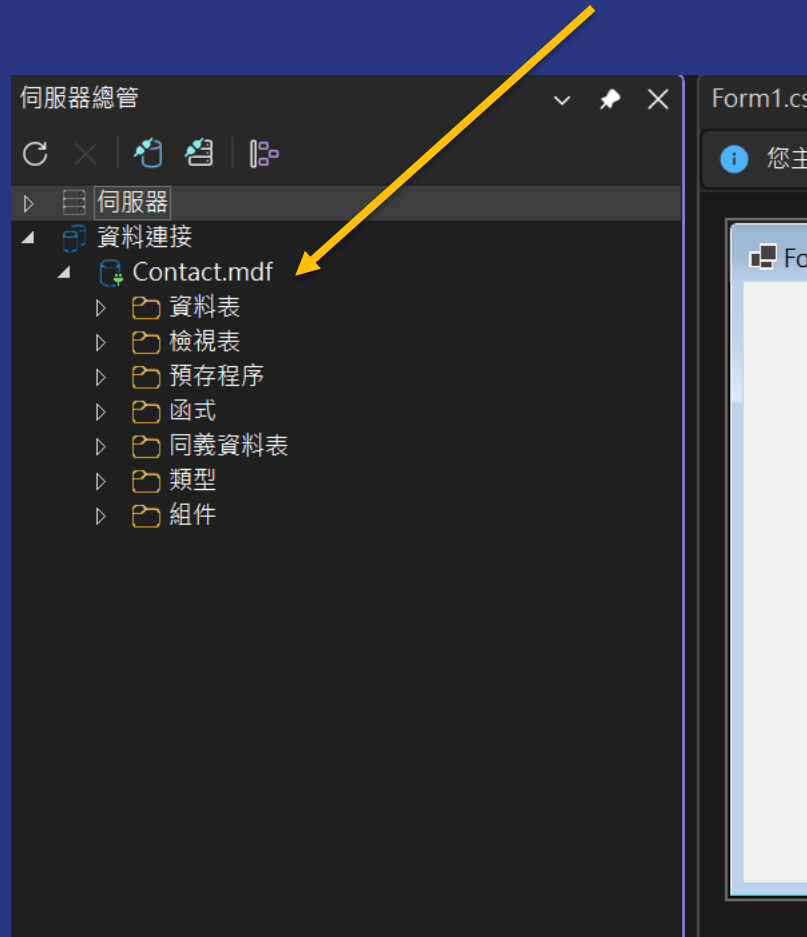
你會發現專案裡面多出了兩個檔案



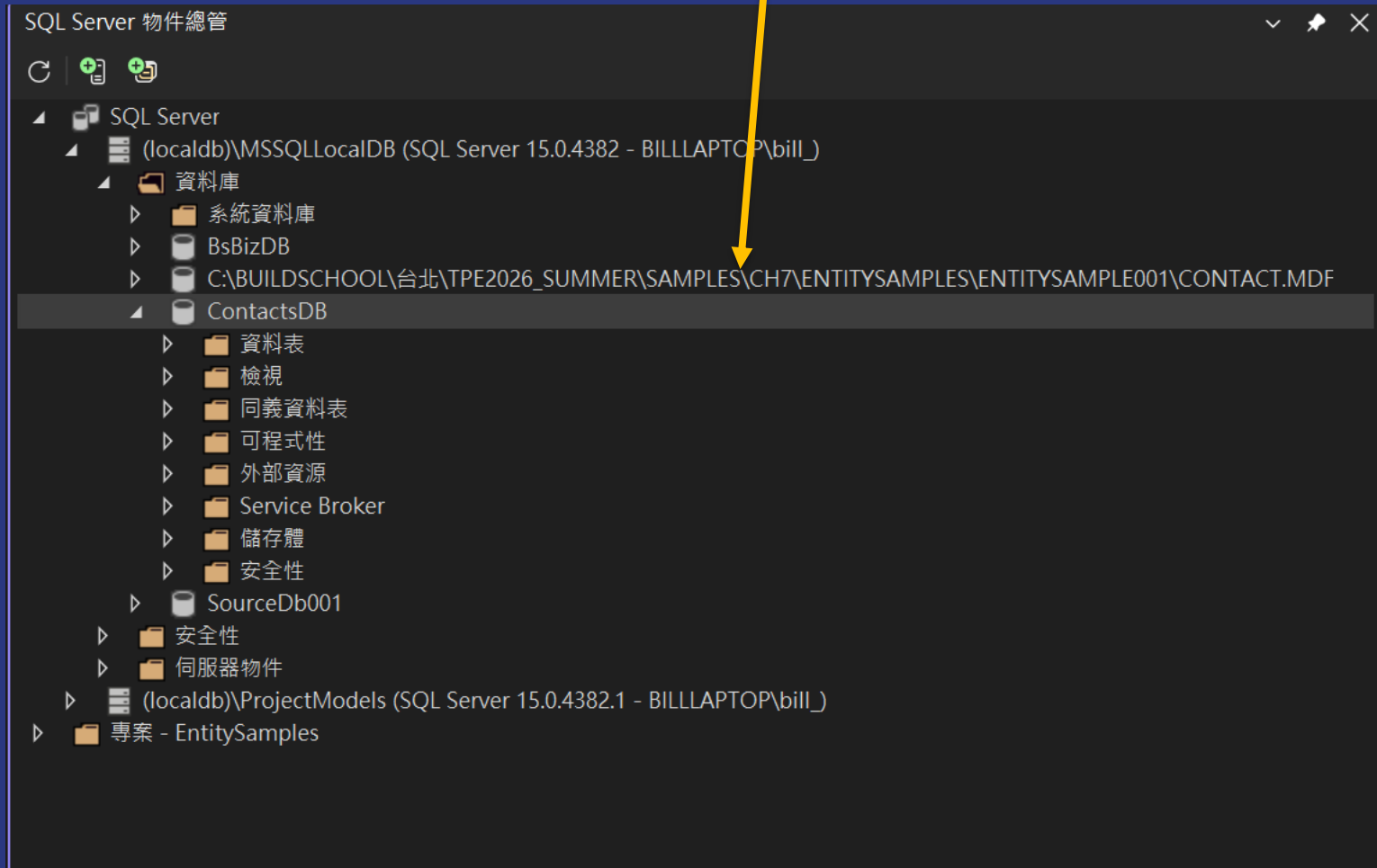
使用檔案總管觀察

☐ 名稱	修改日期	類型	大小
bin	2026/6/11 上午 12:16	檔案資料夾	
obj	2026/6/11 上午 12:16	檔案資料夾	
Contact.mdf	2026/6/11 上午 12:17	MDF 檔案	8,192 KB
Contact_log.ldf	2026/6/11 上午 12:17	LDF 檔案	8,192 KB
EntitySample001.csproj	2026/6/11 上午 12:16	C# Project File	1 KB
EntitySample001.csproj.user	2026/6/11 上午 12:16	Per-User Project Op...	1 KB
Form1.cs	2026/6/11 上午 12:16	C# Source File	1 KB
Form1.Designer.cs	2026/6/11 上午 12:16	C# Source File	2 KB
Form1.resx	2026/6/11 上午 12:16	Microsoft .NET Man...	6 KB
Program.cs	2026/6/11 上午 12:16	C# Source File	1 KB

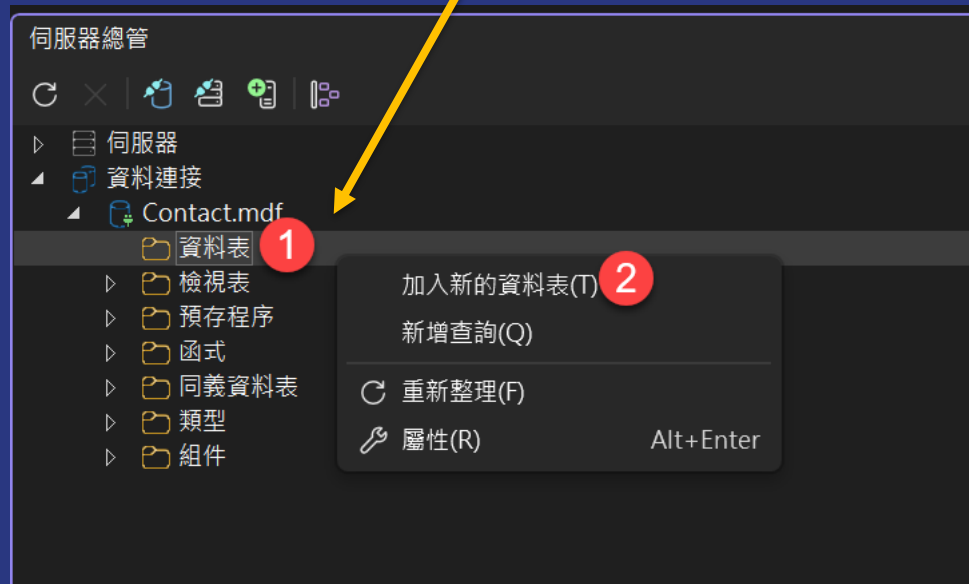
開啟【伺服器總管】，然後展開【資料連接】
也會看到剛剛建立的資料庫



也可以使用 【SQL Server 物件總管】



回到 【伺服器總管】
在【資料表】目錄按滑鼠右鍵
選擇【加入新的資料表】



就出現資料表的設計畫面

The screenshot displays the SQL Server Enterprise Designer interface. The top tab bar shows 'dbo.Table [設計]' and 'Form1.cs [設計]'. Below the tab bar, the '更新(U)' button and '指令碼檔案(I):' dropdown (set to 'dbo.Table.sql') are visible. The main workspace is divided into two panes. The left pane shows the table design grid with columns 'Id' (int, NOT NULL, PRIMARY KEY) and an unnamed column (int, NULL, PRIMARY KEY). The right pane shows the '金鑰 (1)' (Keys) section, indicating a primary key for the unnamed column. Below the keys section, there are sections for '檢查條件約束 (0)' (Constraints), '索引 (0)' (Indexes), '外部索引鍵 (0)' (Foreign Keys), and '觸發程序 (0)' (Triggers). The bottom pane shows the T-SQL view with the following code:

```
1 CREATE TABLE [dbo].[Table]
2 (
3     [Id] INT NOT NULL PRIMARY KEY
4 )
5
```


選擇資料欄位，打開屬性頁籤

The screenshot displays the SQL Server Enterprise Designer interface. The main window shows the 'dbo.Table' design view with a table structure. The 'Id' field is selected, and the 'Properties' window is open on the right, showing the 'Id' field's properties.

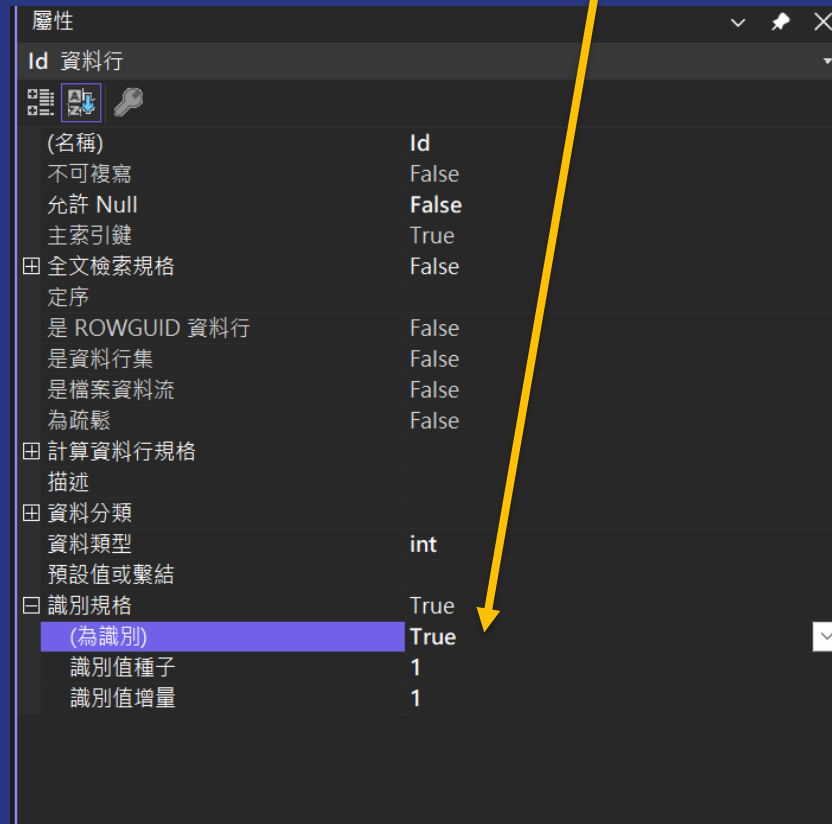
Table Structure:

名稱	資料類型	允許 Null	預設
Id	int	☐	

Properties Window (Id 資料行):

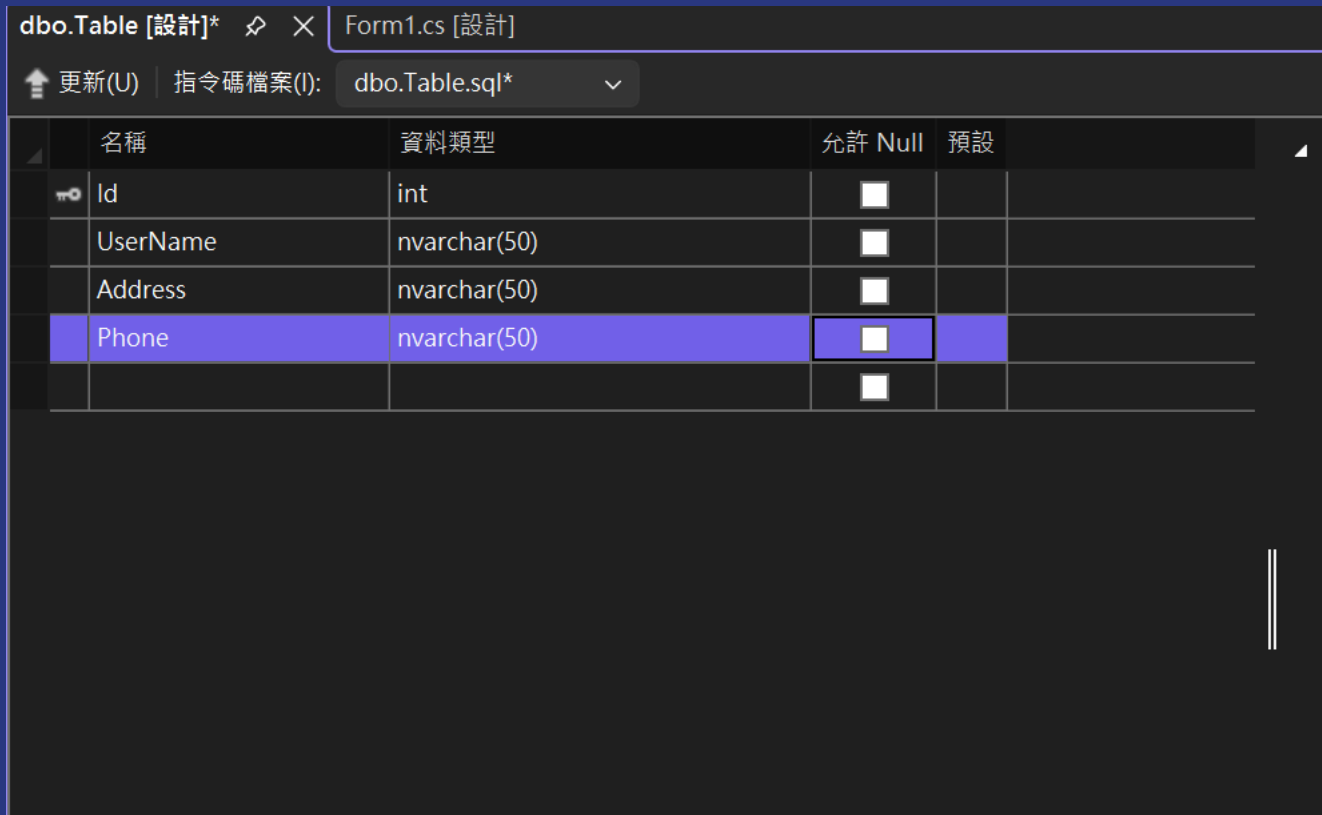
屬性	值
(名稱)	Id
不可複寫	False
允許 Null	False
主索引鍵	True
全文檢索規格	False
定序	
是 ROWGUID 資料行	False
是資料行集	False
是檔案資料流	False
為疏鬆	False
計算資料行規格	
描述	
資料分類	
資料類型	int
預設值或繫結	
識別規格	False

將 Id 這個欄位(資料行)的為識別(Is Identity) 設定為 true



這個設定啟用讓此資料行變成自動增量，也就是你不必處理這個資料行，你每新增一筆資料，系統就自動幫你+1

按照下圖的樣式，新增其他資料行

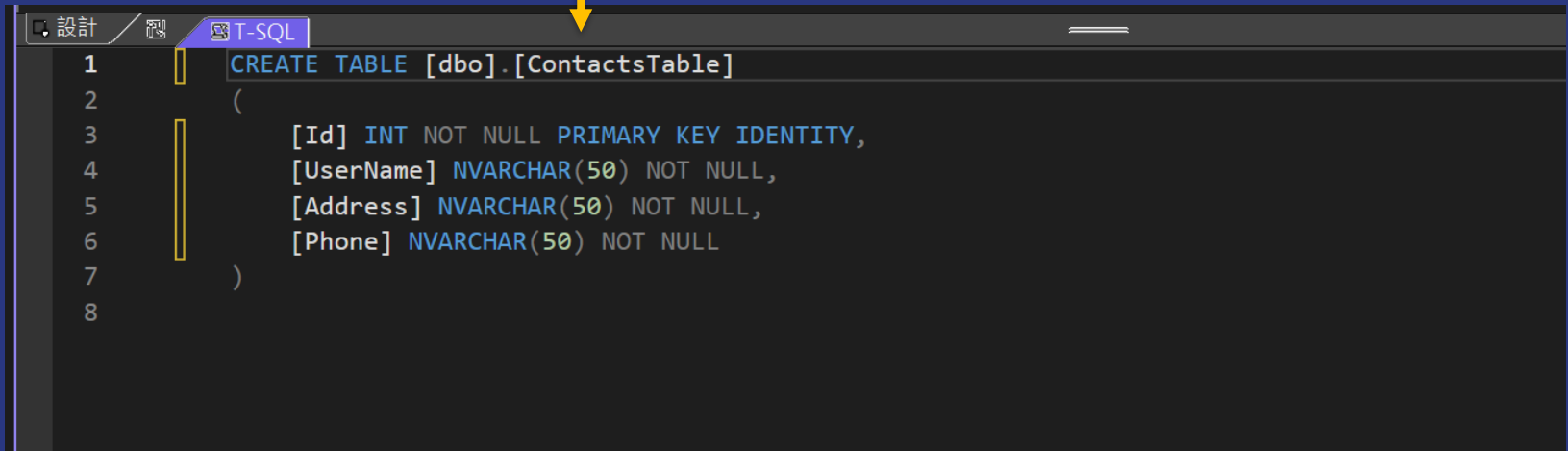


The screenshot shows a database table design interface. The table is named 'dbo.Table' and is in design mode. The columns are: Id (int, primary key), UserName (nvarchar(50)), Address (nvarchar(50)), and Phone (nvarchar(50)). The 'Phone' row is highlighted in blue. The 'Id' row has a key icon, indicating it is the primary key. The interface includes a toolbar with '更新(U)' (Update) and '指令碼檔案(I):' (Script File) buttons, and a dropdown menu showing 'dbo.Table.sql*'. The table structure is as follows:

	名稱	資料類型	允許 Null	預設
key	Id	int	☐	
	UserName	nvarchar(50)	☐	
	Address	nvarchar(50)	☐	
	Phone	nvarchar(50)	☐	
			☐	

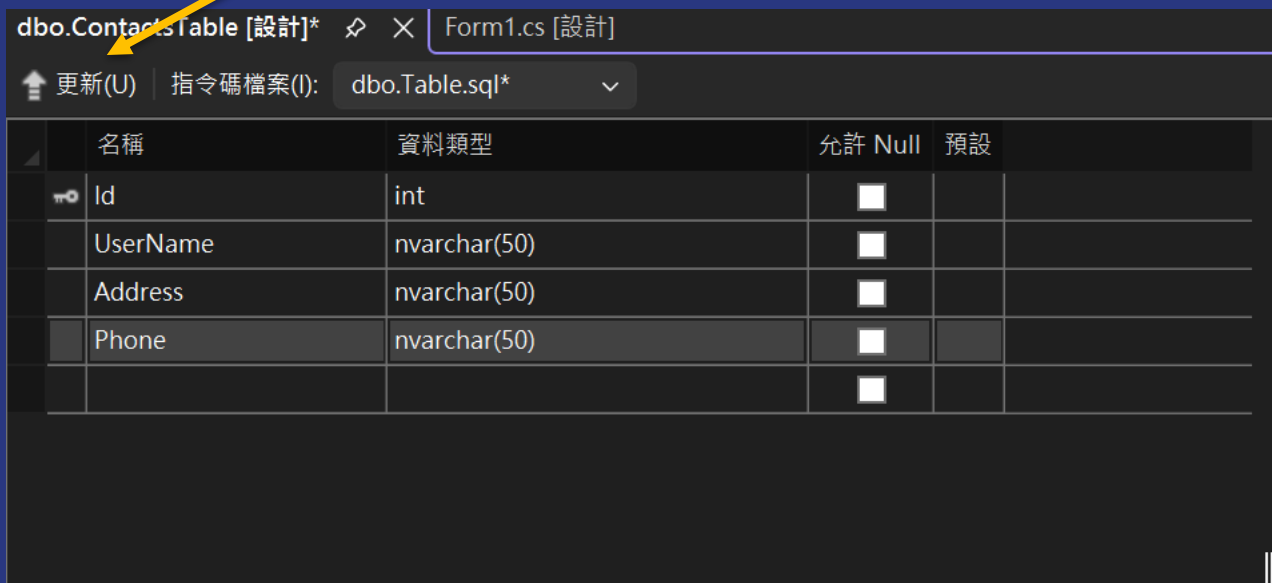
id 資料行前方有個鑰匙符號，表示為主索引

在 T-SQL窗格把資料表名稱改為 ContactsTable

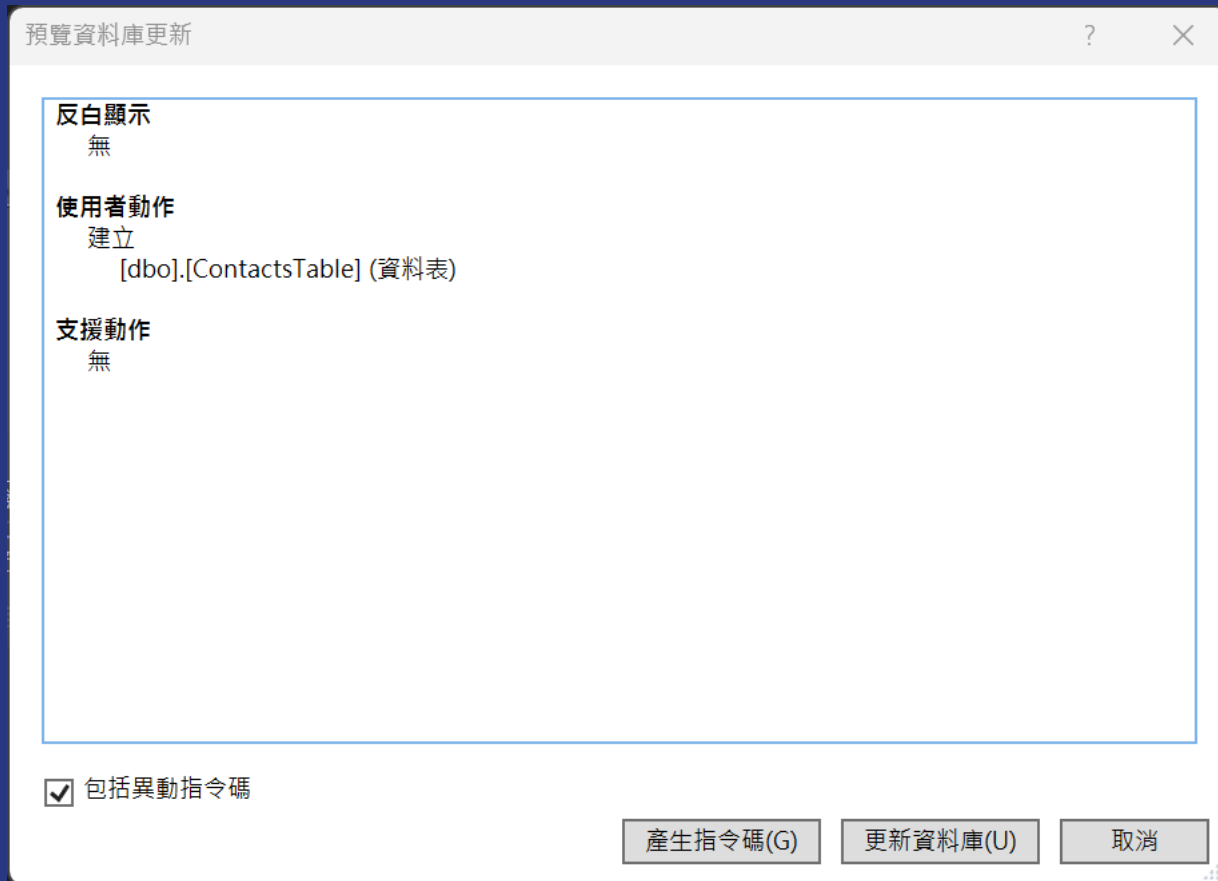


```
1 CREATE TABLE [dbo].[ContactsTable]
2 (
3     [Id] INT NOT NULL PRIMARY KEY IDENTITY,
4     [UserName] NVARCHAR(50) NOT NULL,
5     [Address] NVARCHAR(50) NOT NULL,
6     [Phone] NVARCHAR(50) NOT NULL
7 )
8
```

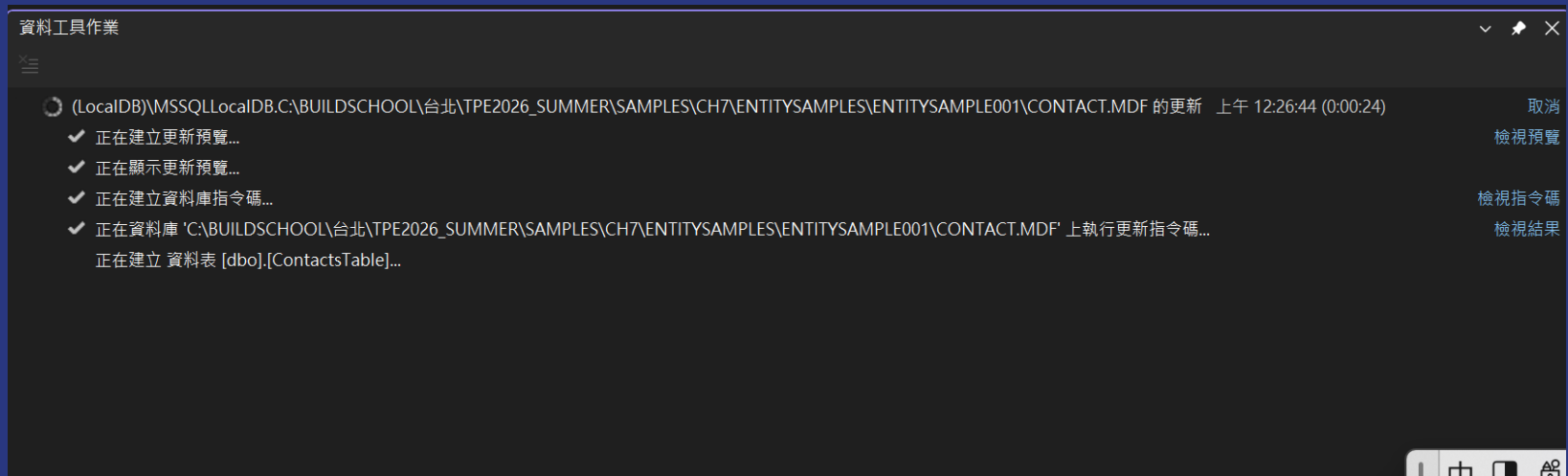
完成設計後，按下 【更新】 (Update)



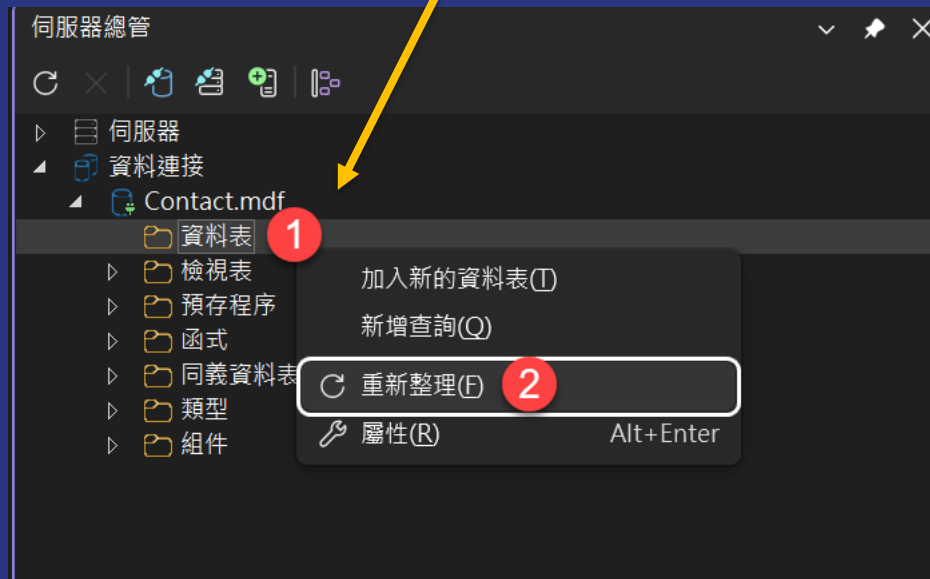
出現提示視窗，確認使用者動作無誤後，
按下【更新資料庫】



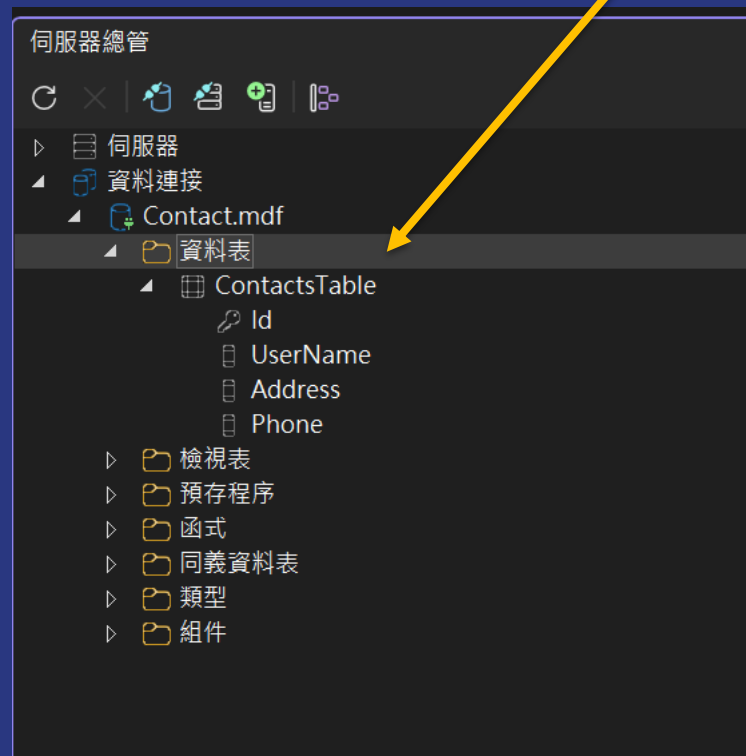
資料工具作業視窗會顯示進度的訊息



回到【伺服器總管】，重新整理資料表



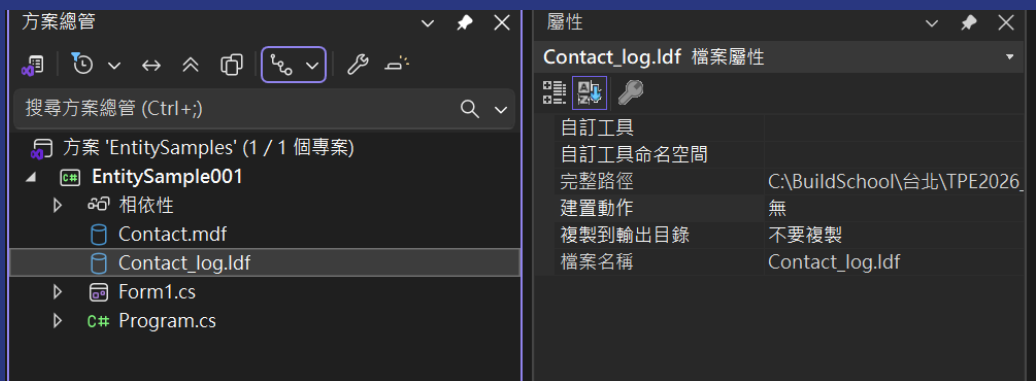
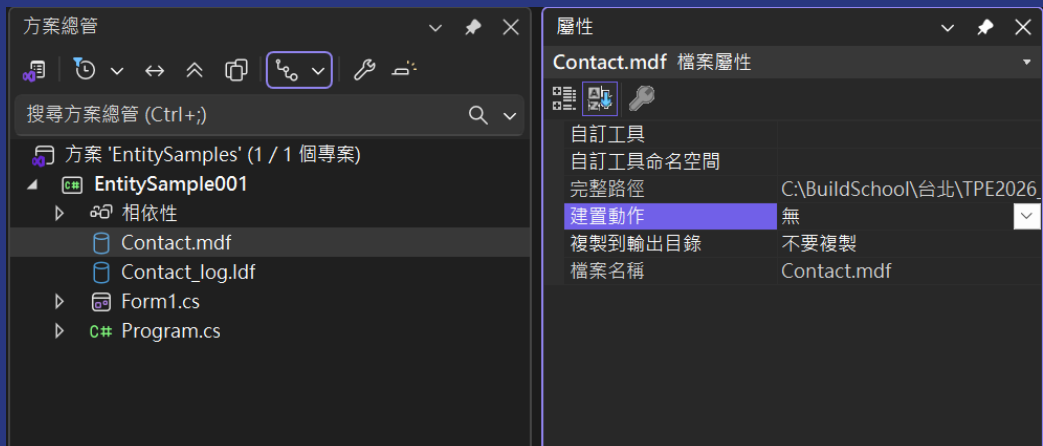
如果看到【ContactsTable】就成功了



確認 Contact.mdf 與 Contact_log.ldf 的檔案屬性為

(1) 建置動作：無

(2) 複製到輸出目錄：不要複製



如果新增 local db 失敗

1. 強制停止 `sqllocaldb stop mssqllocaldb -k`
2. 刪除 `sqllocaldb delete mssqllocaldb`
3. 重建 `sqllocaldb create mssqllocaldb`
4. 啟動 `sqllocaldb start mssqllocaldb`

LAB

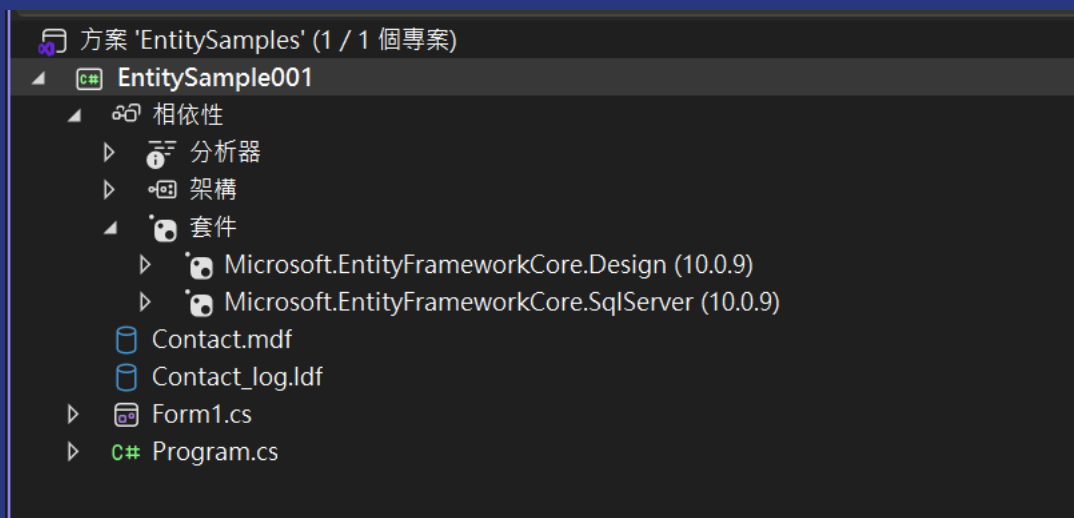
Code First From DB
(繼續上面的專案)

Code first from db

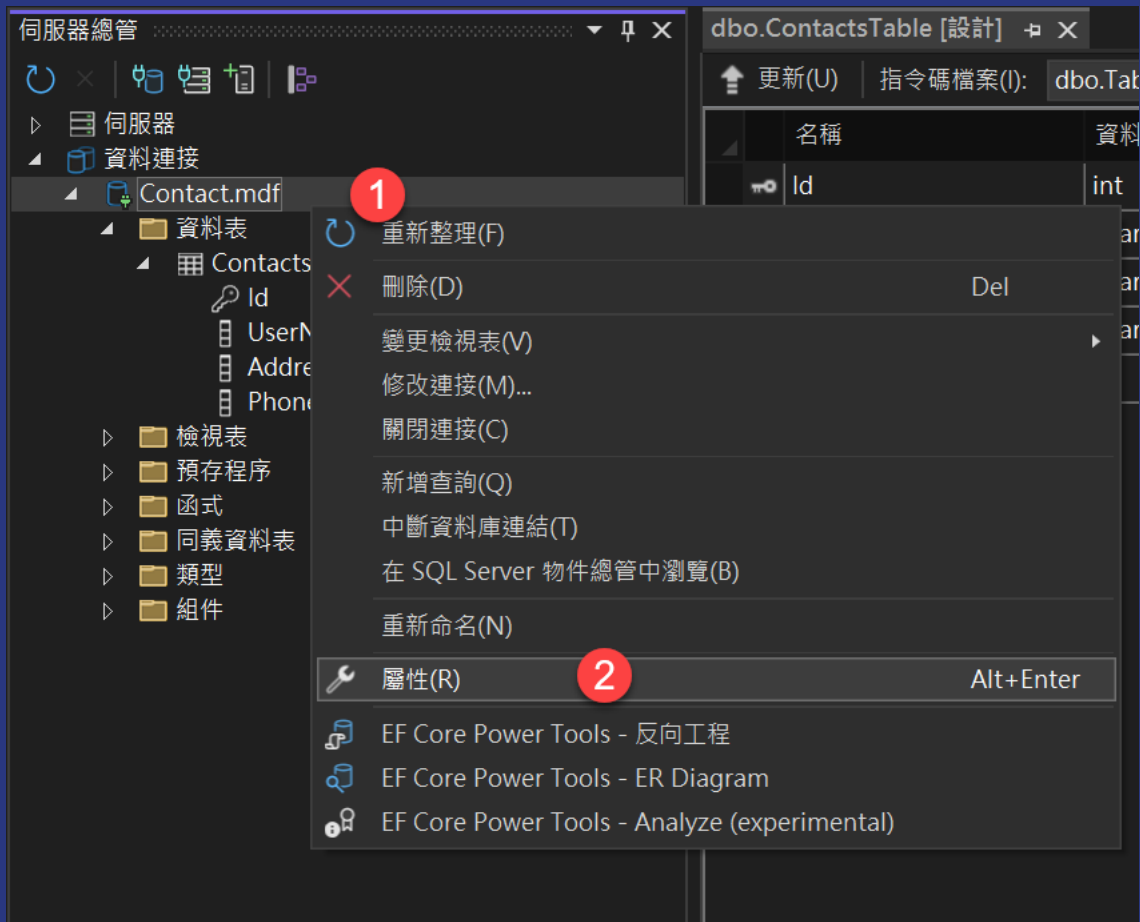
1. 手動建立 `local db` 與 資料表
 - 姑且稱為『範本資料庫』
2. 透過工具 (`dotnet ef cli tool`) 產出 `DbContext` 與 `entities (models)`
3. 加入連線字串檔案 (`appsettings.json`)
4. 執行時期透過 `C# code` 產出執行時期使用的資料庫
 - 姑且稱為『正式資料庫』

安裝必要的 nuget package

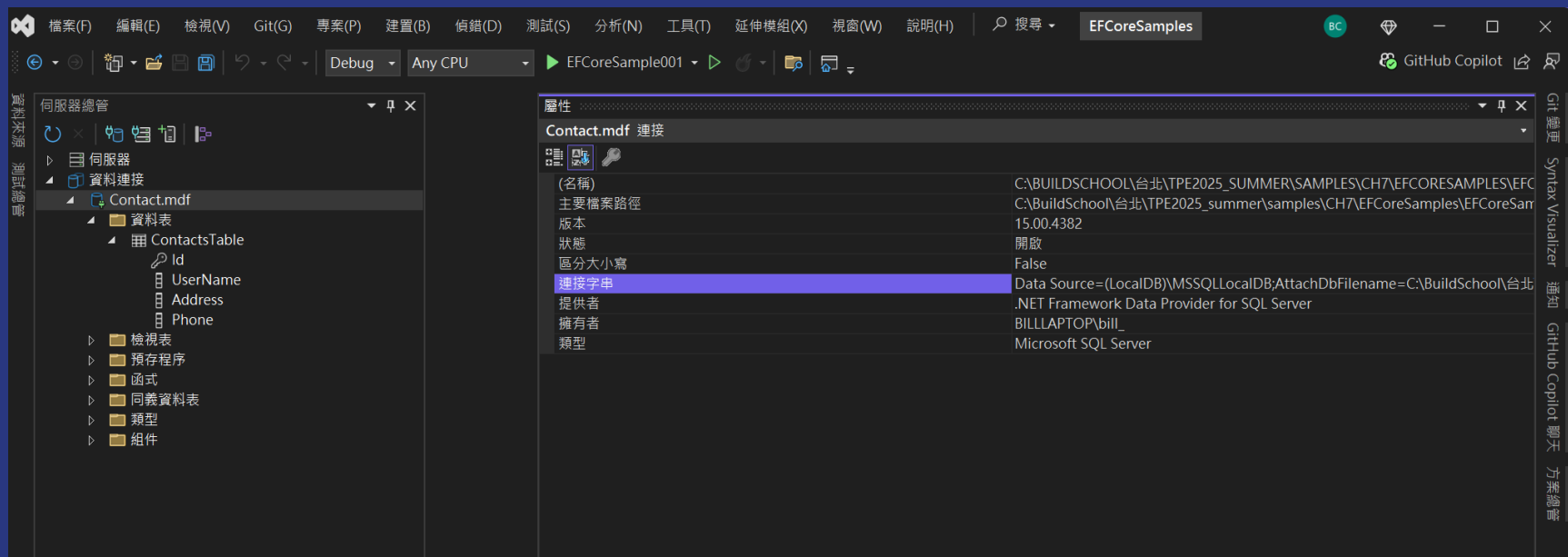
- Microsoft.EntityFrameworkCore.Design
- Microsoft.EntityFrameworkCore.SqlServer
- 版本選最新穩定版
 - 注意詳細資料的相依性



使用伺服器總管 取得『範本資料庫』連線字串



複製連線字串內容，代入到【dotnet ef dbcontext scaffold “連線字串” ...】
(完整命令參考 Scripts.txt 內容)



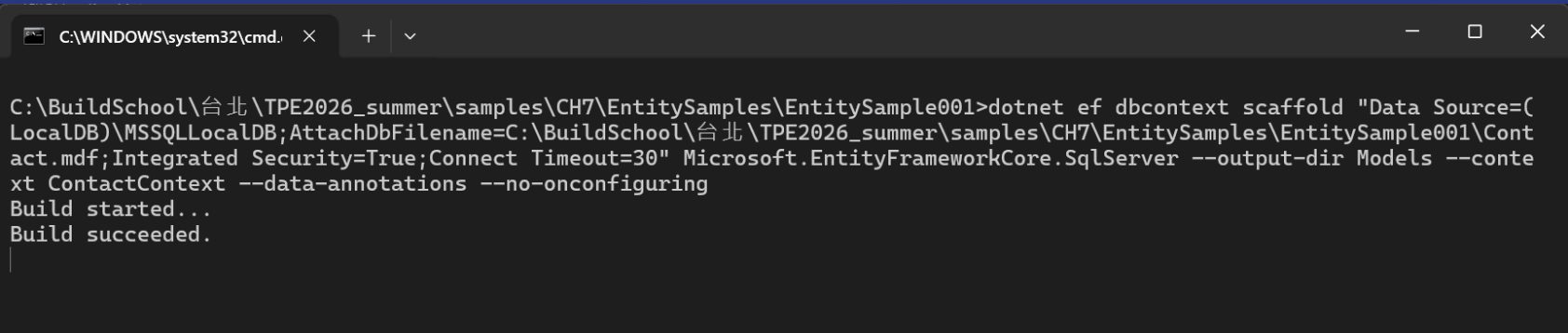
執行 powershell 命令

1. 切換到專案目錄

- `cd EFCoreSample001`

2. 執行 `dotnet ef dbcontext scaffold` “連線字串” ...】

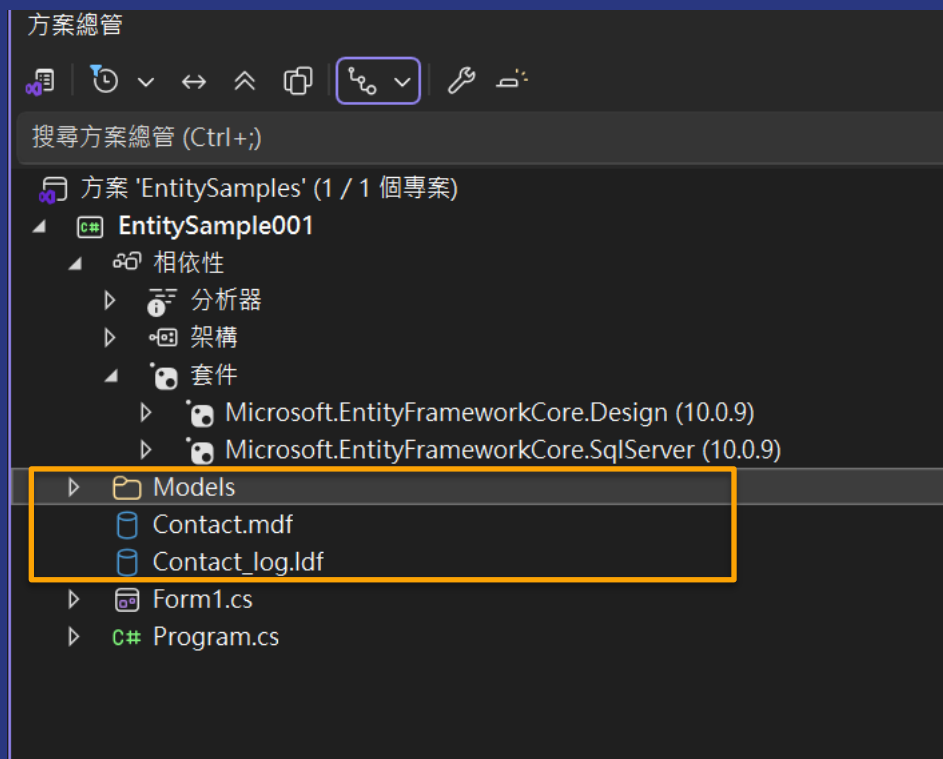
- 完整命令參考 `Scripts.txt` 內容



```
C:\WINDOWS\system32\cmd.  X  +  v

C:\BuildSchool\台北\TPE2026_summer\samples\CH7\EntitySamples\EntitySample001>dotnet ef dbcontext scaffold "Data Source=(
LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\BuildSchool\台北\TPE2026_summer\samples\CH7\EntitySamples\EntitySample001\Cont
act.mdf;Integrated Security=True;Connect Timeout=30" Microsoft.EntityFrameworkCore.SqlServer --output-dir Models --conte
xt ContactContext --data-annotations --no-onconfiguring
Build started...
Build succeeded.
```

成功建立後會出現 Models 資料夾
並包含產出的檔案



ContactsTable class

- 這個類別就是依據你的資料表建立的 C# 類別，被稱為 ContactsTable 的 model class

```
[Table("ContactsTable")]
public partial class ContactsTable
{
    [Key]
    public int Id { get; set; }

    [StringLength(50)]
    public string UserName { get; set; } = null!;

    [StringLength(50)]
    public string Address { get; set; } = null!;

    [StringLength(50)]
    public string Phone { get; set; } = null!;
}
```

ContactContext class

- 這個類別就是負責存取 Contacts 資料庫的類別，其父類別為 DbContext class

```
public partial class ContactContext : DbContext
{
    public ContactContext(DbContextOptions<ContactContext> options)
        : base(options)
    {
    }

    public virtual DbSet<ContactsTable> ContactsTables { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<ContactsTable>(entity =>
        {
            entity.HasKey(e => e.Id).HasName("PK__Contacts__3214EC07251690FA");
            entity.Property(e => e.Address).UseCollation("SQL_Latin1_General_CP1_CI_AS");
            entity.Property(e => e.Phone).UseCollation("SQL_Latin1_General_CP1_CI_AS");
            entity.Property(e => e.UserName).UseCollation("SQL_Latin1_General_CP1_CI_AS");
        });

        modelBuilder.OnModelCreatingPartial(modelBuilder);
    }

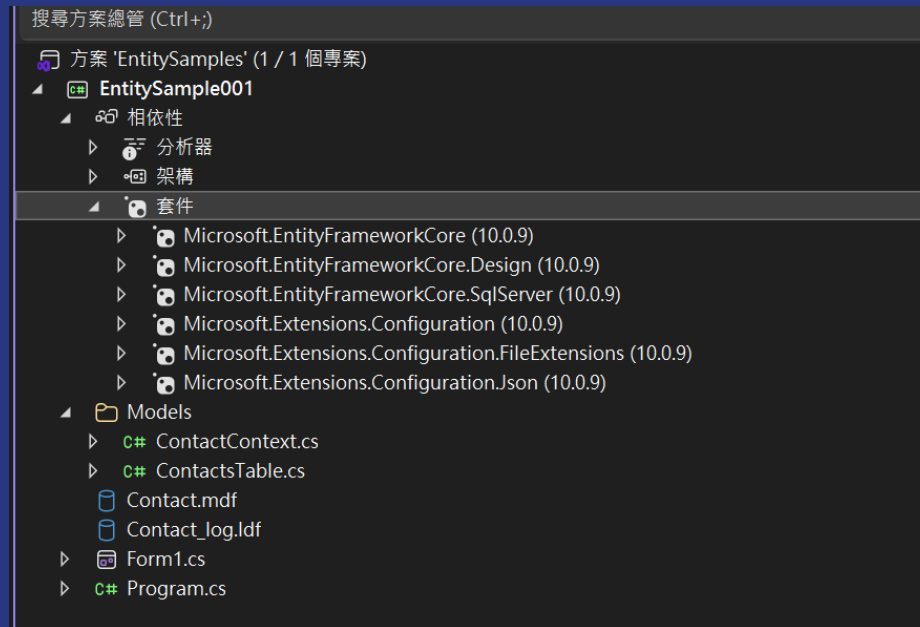
    partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}
```

LAB

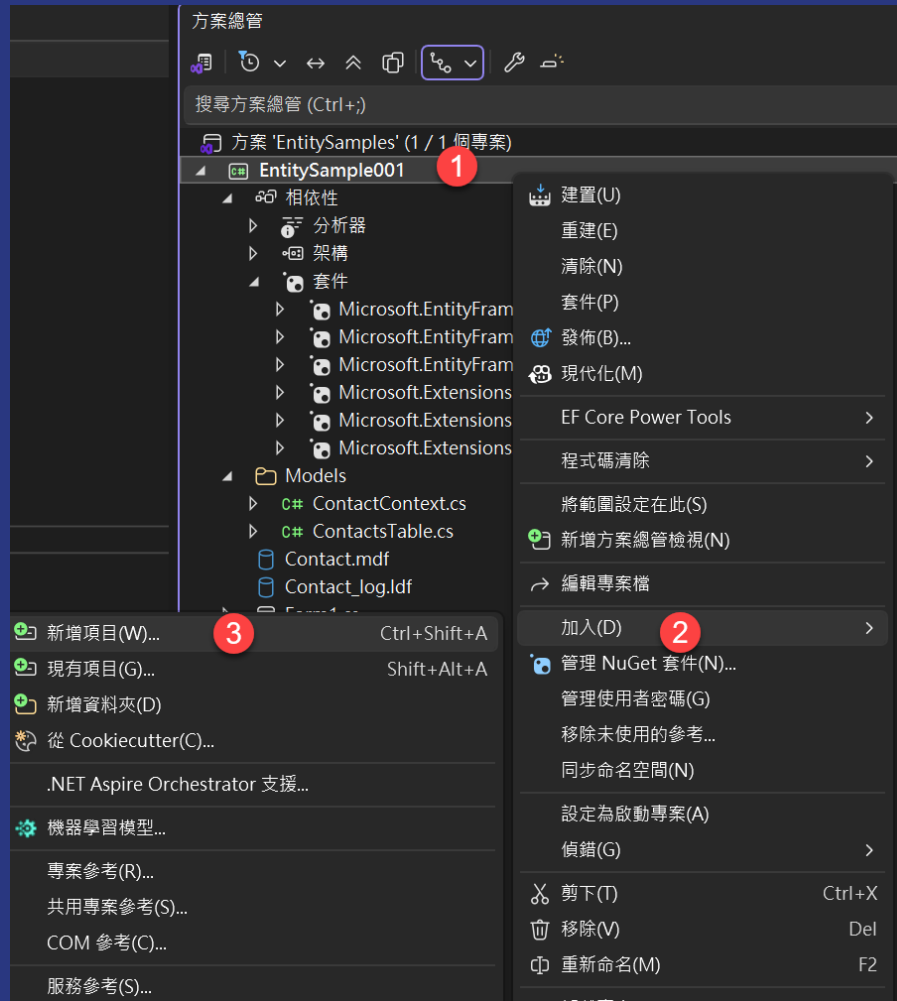
讓程式碼產出正式資料庫
(繼續上面的專案)

安裝必要的 nuget package

- Microsoft.EntityFrameworkCore
- Microsoft.Extensions.Configuration
- Microsoft.Extensions.Configuration.FileExtensions
- Microsoft.Extensions.Configuration.Json
- 版本選最新穩定版



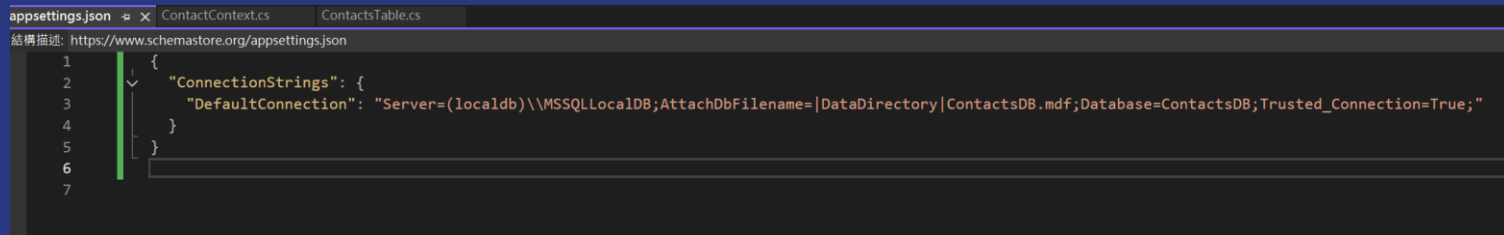
加入 appsettings.json



選【Web】【標記】【JSON 檔案】
命名為 appsettings.json



在 appsettings.json 加入正式資料庫連線字串
(完整內容參考 Scripts.txt 內容)

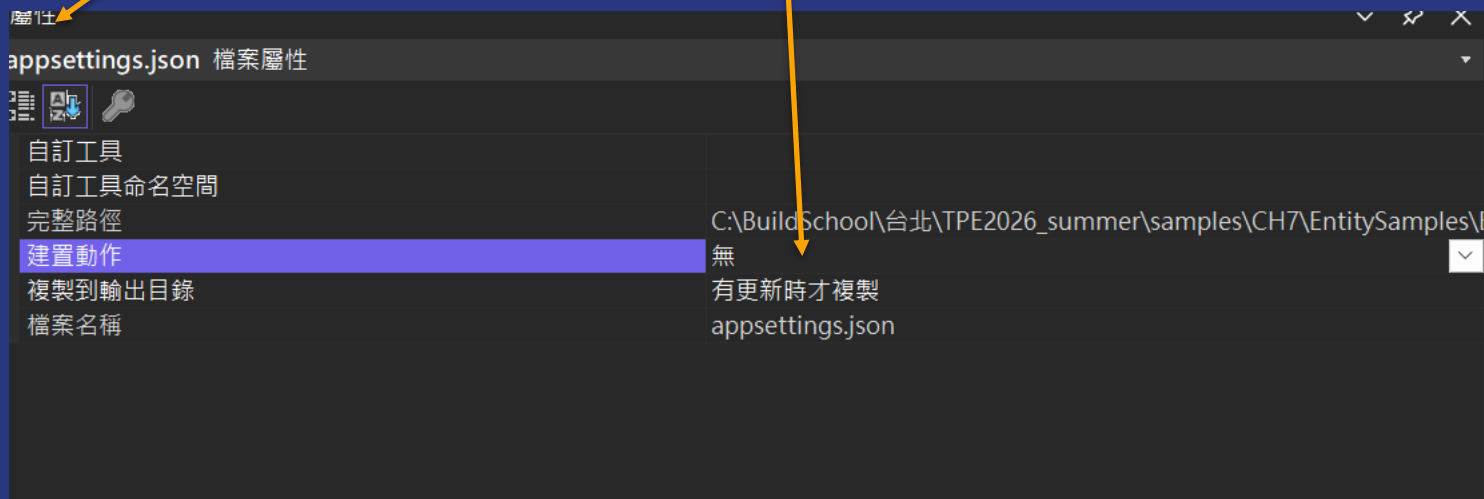


The screenshot shows a code editor with three tabs: `appsettings.json`, `ContactContext.cs`, and `ContactsTable.cs`. The `appsettings.json` tab is active, displaying the following JSON configuration:

```
1 {  
2   "ConnectionStrings": {  
3     "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;AttachDbFilename=|DataDirectory|ContactsDB.mdf;Database=ContactsDB;Trusted_Connection=True;"  
4   }  
5 }  
6  
7
```

Below the code, a link for the schema description is provided: [結構描述: https://www.schemastore.org/appsettings.json](https://www.schemastore.org/appsettings.json).

把 appsettings.json 的【複製到輸出目錄】
設定為“有更新才複製”



在 Program class 建立靜態屬性 (存取設定)

```
internal static class Program
{
    public static IConfiguration? Configuration { get; private set; }
```

在 Program class 建立初始化資料庫關程序

```
static void LoadConfiguration()
{
    Configuration = new ConfigurationBuilder()
        .SetBasePath(AppDomain.CurrentDomain.BaseDirectory)
        .AddJsonFile("appsettings.json")
        .Build();
}
public static void InitialDB()
{
    var optionsBuilder = new DbContextOptionsBuilder<ContactContext>();
    optionsBuilder.UseSqlServer(Configuration?.GetConnectionString("DefaultConnection"));

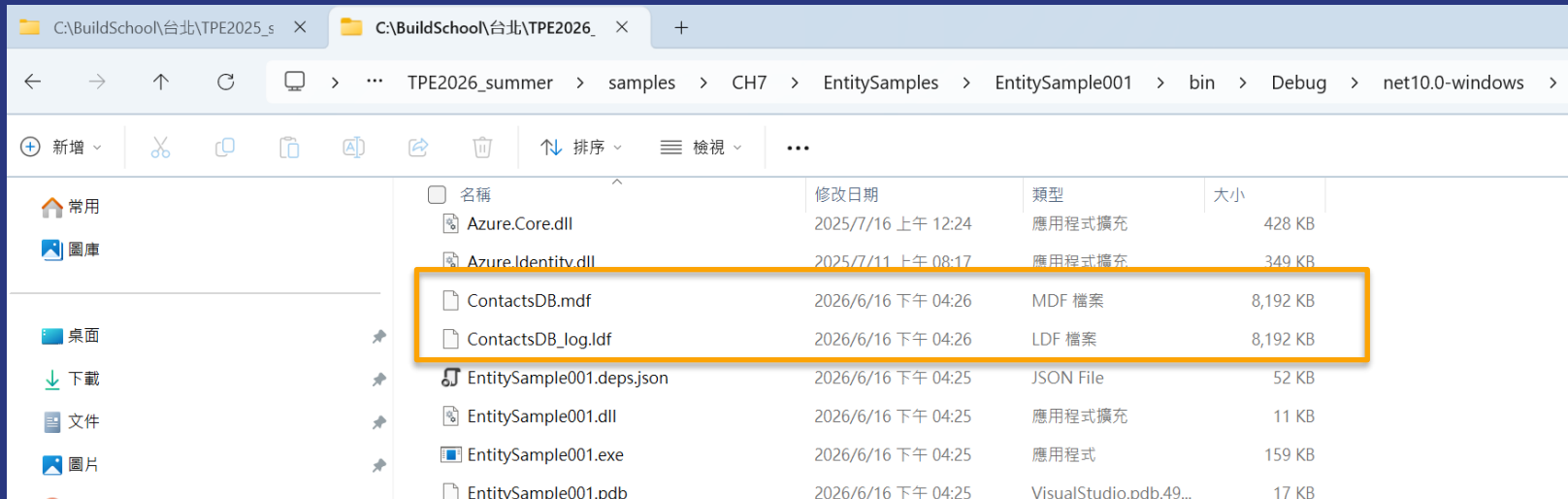
    using (var context = new ContactContext(optionsBuilder.Options))
    {
        context.Database.EnsureCreated();
    }
}
```

修改 Program.Main 方法，呼叫必要的初始化

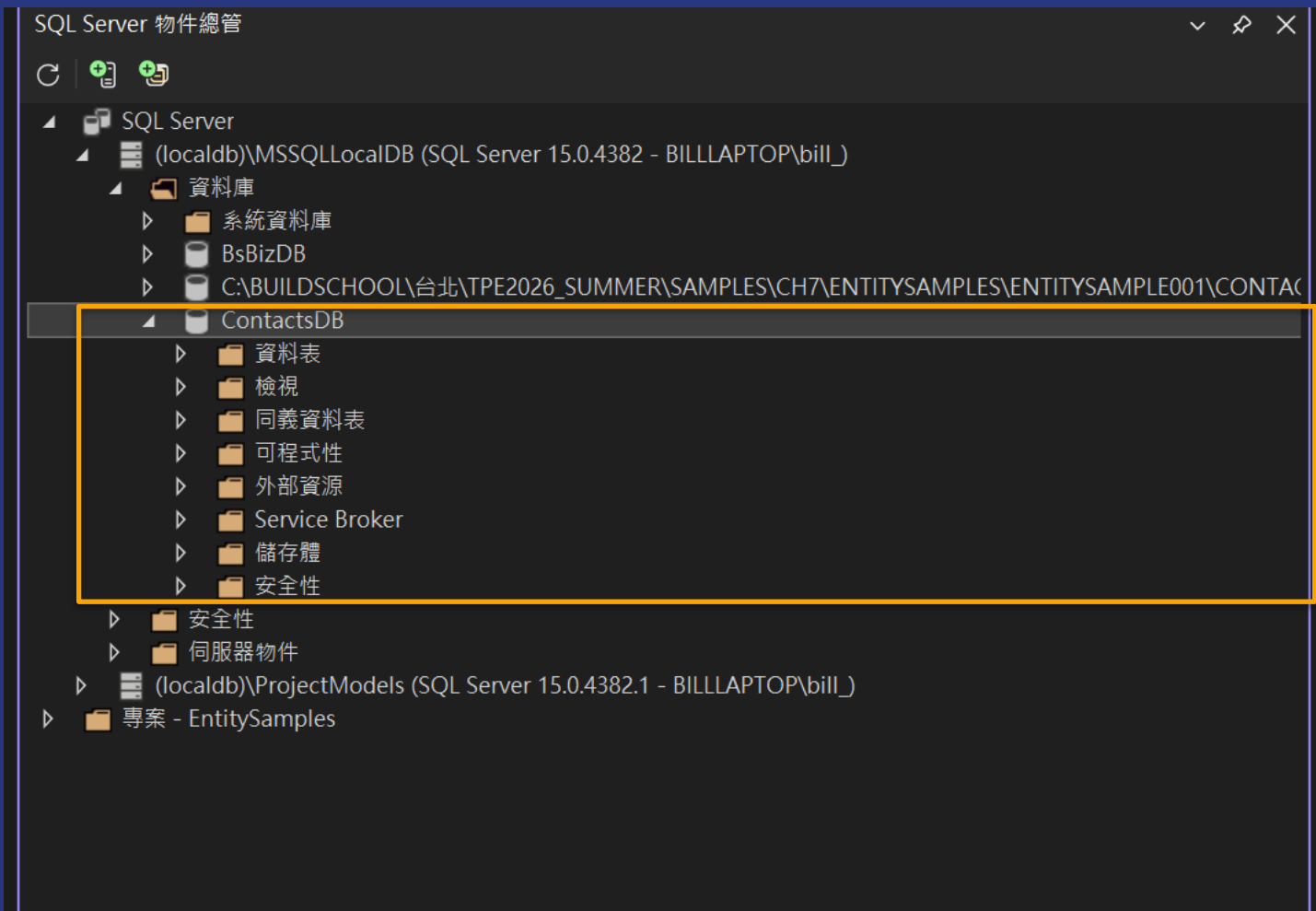
```
static void Main()
{
    ApplicationConfiguration.Initialize();
    LoadConfiguration();
    InitialDB();
    Application.Run(new Form1());
}
```

執行程式，確認正式資料庫有建立

1. 使用檔案總管觀察 exe 檔案所在目錄



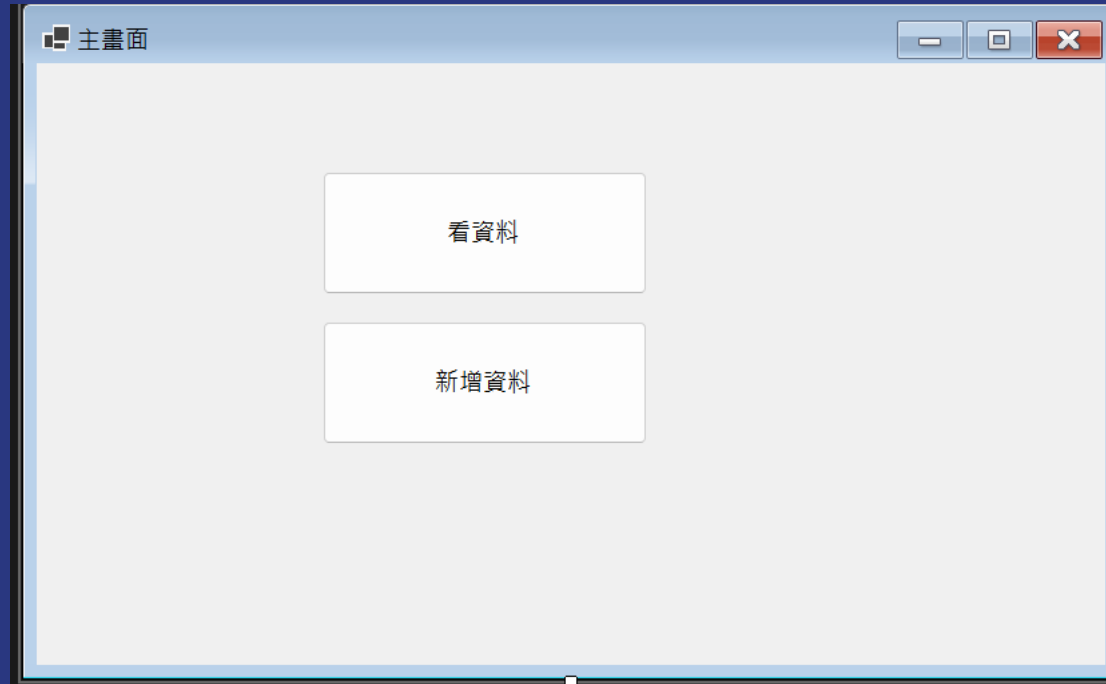
2. 使用 SQL Server 物件總管



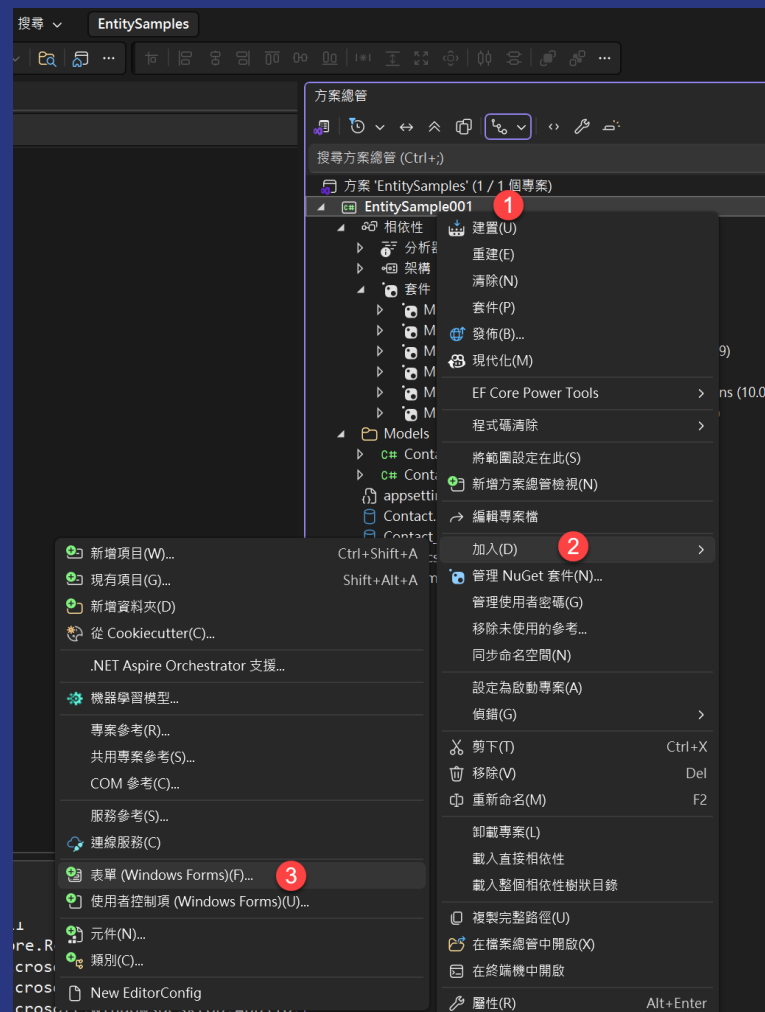
LAB

完成簡單的資料存取程式
(繼續上面的專案)

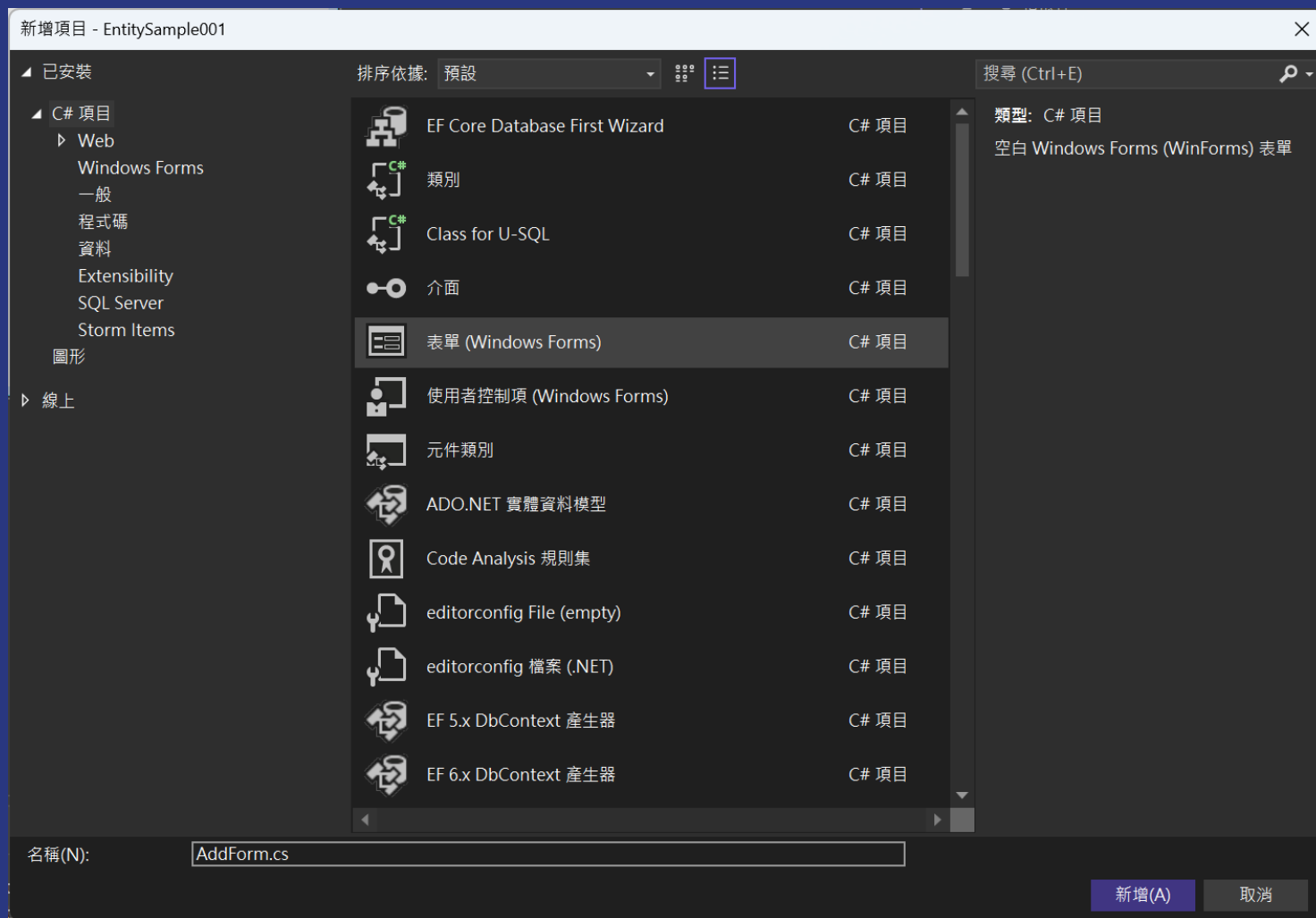
在 Form1 上加入兩個 Button
(1) 看資料 (2) 新增資料



我們加入一個新的 Form，
在專案上按【加入】→【Windows Form】



新的 Form 請命名為 AddForm

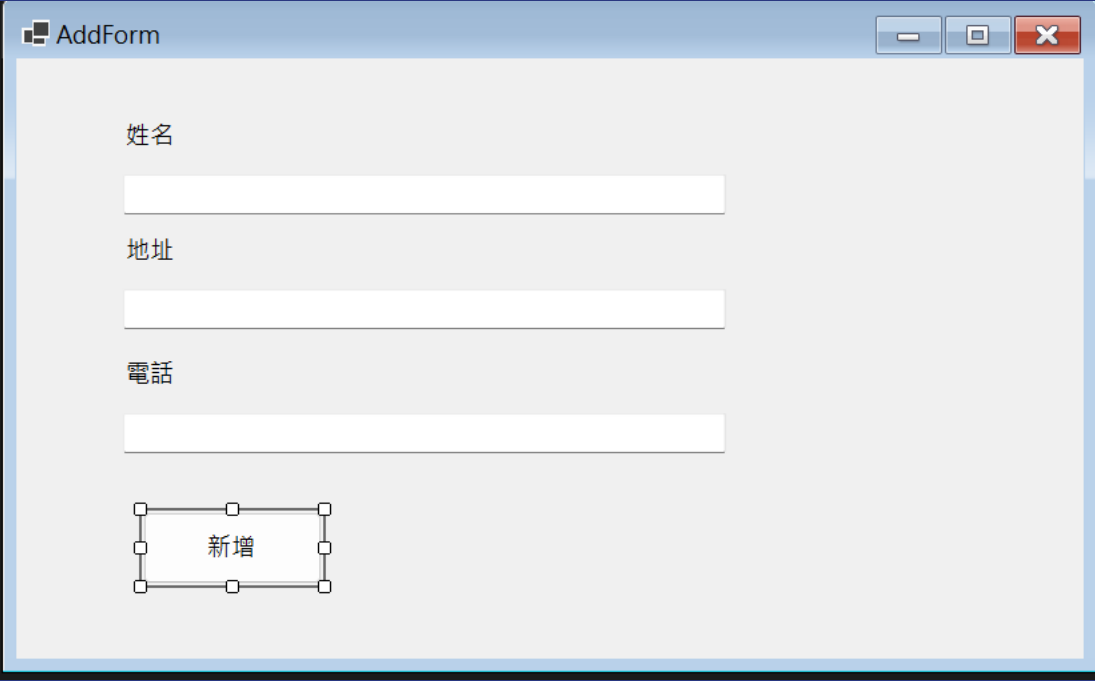


AddForm 的畫面配置

三個 Lable

三個 TextBox

一個 Button



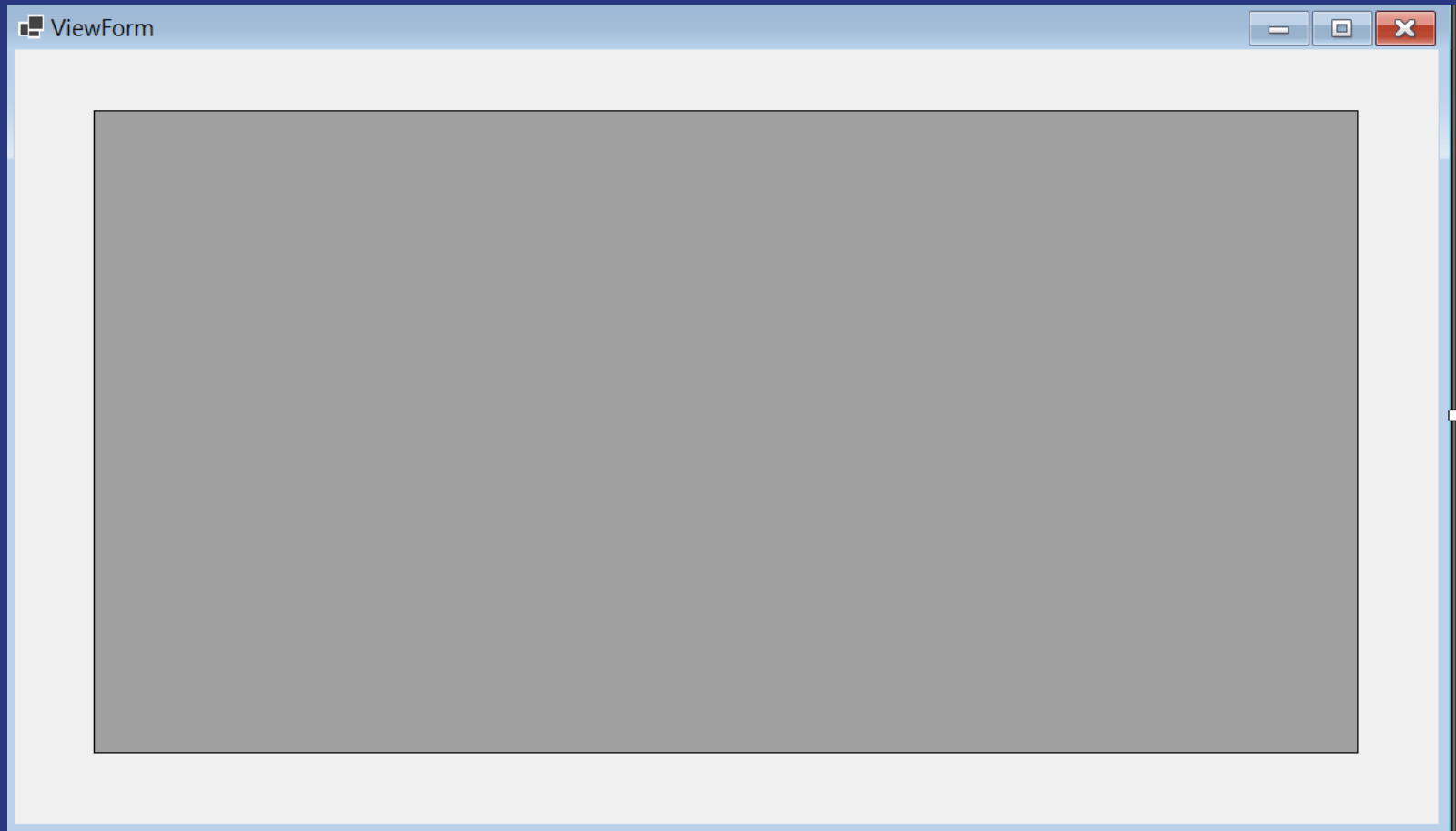
The screenshot shows a Windows-style window titled "AddForm". Inside the window, there is a form with three labels: "姓名" (Name), "地址" (Address), and "電話" (Phone). Each label is followed by a text box. At the bottom of the form, there is a button labeled "新增" (Add). The button has a rectangular border with small squares at the corners and midpoints, indicating it is a standard Windows button.

修改 AddForm class 為 Button.Click 事件掛上委派方法

```
private void button1_Click(object sender, EventArgs e)
{
    ContactsTable data = new ContactsTable()
    {
        UserName = textBox1.Text,
        Address = textBox2.Text,
        Phone = textBox3.Text
    };
    try
    {
        var optionsBuilder = new DbContextOptionsBuilder<ContactContext>();
        optionsBuilder.UseSqlServer(Program.Configuration?.GetConnectionString("DefaultConnection"));
        using (var context = new ContactContext(optionsBuilder.Options))
        {
            context.ContactsTables.Add(data);
            context.SaveChanges();
        }
        MessageBox.Show("新增成功");
        ClearTextBox();
    }
    catch (Exception ex)
    {
        MessageBox.Show("新增失敗:" + ex.Message);
    }
}

private void ClearTextBox()
{
    textBox1.Clear();
    textBox2.Clear();
    textBox3.Clear();
}
```

新增一個 ViewForm 並擺上一個 DataGridView



修改 ViewForm class

```
public partial class ViewForm : Form
{
    public ViewForm()
    {
        InitializeComponent();
        BindData();
    }

    private void BindData()
    {
        var optionsBuilder = new DbContextOptionsBuilder<ContactContext>();
        optionsBuilder.UseSqlServer(Program.Configuration?.GetConnectionString("DefaultConnection"));
        using (var context = new ContactContext(optionsBuilder.Options))
        {
            var data = context.ContactsTables.ToList();
            dataGridView1.DataSource = data;
        }
    }
}
```

回到 Form1 class ,
為兩個 Button 分別掛上 Click 事件的委派方法

```
private void button1_Click(object sender, EventArgs e)
{
    var form = new ViewForm();
    form.Show();
}

private void button2_Click(object sender, EventArgs e)
{
    var form = new AddForm();
    form.ShowDialog();
}
```

Show() 和 ShowDialog() 有甚麼不同？



執行

再談建構式

建構式 constructor

- 類別 或 結構 建立時，會呼叫它的建構函式。 建構函式的名稱與類別或結構相同，因此，它們通常用來初始化新物件的資料成員
- 不使用任何參數的建構函式稱為「預設建構函式」(Default Constructor)。 每當使用 `new` 運算子來具現化物件，而且未提供引數給 `new` 時，便會叫用預設建構函式
- 建構式也可以多載
- 建構式不會繼承，但需要呼叫父類別的建構式
- 抽象類別的建構式通常為 `protected`

預設建構式

```
public class MyClass
{
    private int x = 0;
    private int y = 0;
}
```

當你沒有寫任何建構式的時候，編譯器在編譯的時候會幫你做一個出來，所以上面的程式碼就會變成

```
public class MyClass
{
    private int x;
    private int y;

    public MyClass() : base()
    {
        x = 0;
        y = 0;
    }
}
```

這個表示呼叫父類別的無參數建構式

```
public class MyClass
{
    private int x;
    private int y;

    public MyClass() : base()
    {
        x = 0;
        y = 0;
    }
}
```



呼叫父類別的無參數建構式可採隱含呼叫寫法(就是不寫啦)

```
public MyClass()
{
    x = 0;
    y = 0;
}
```

建構式多載

```
public class BaseClass
{
    public int X { get; private set; }

    public BaseClass ()
    {
        X = 0;
    }

    public BaseClass (int y)
    {
        X = y;
    }
}
```

前一頁的程式這樣寫比較漂亮

```
public class BaseClass
{
    public int X { get; private set; }

    public BaseClass() : this(0)
    { }

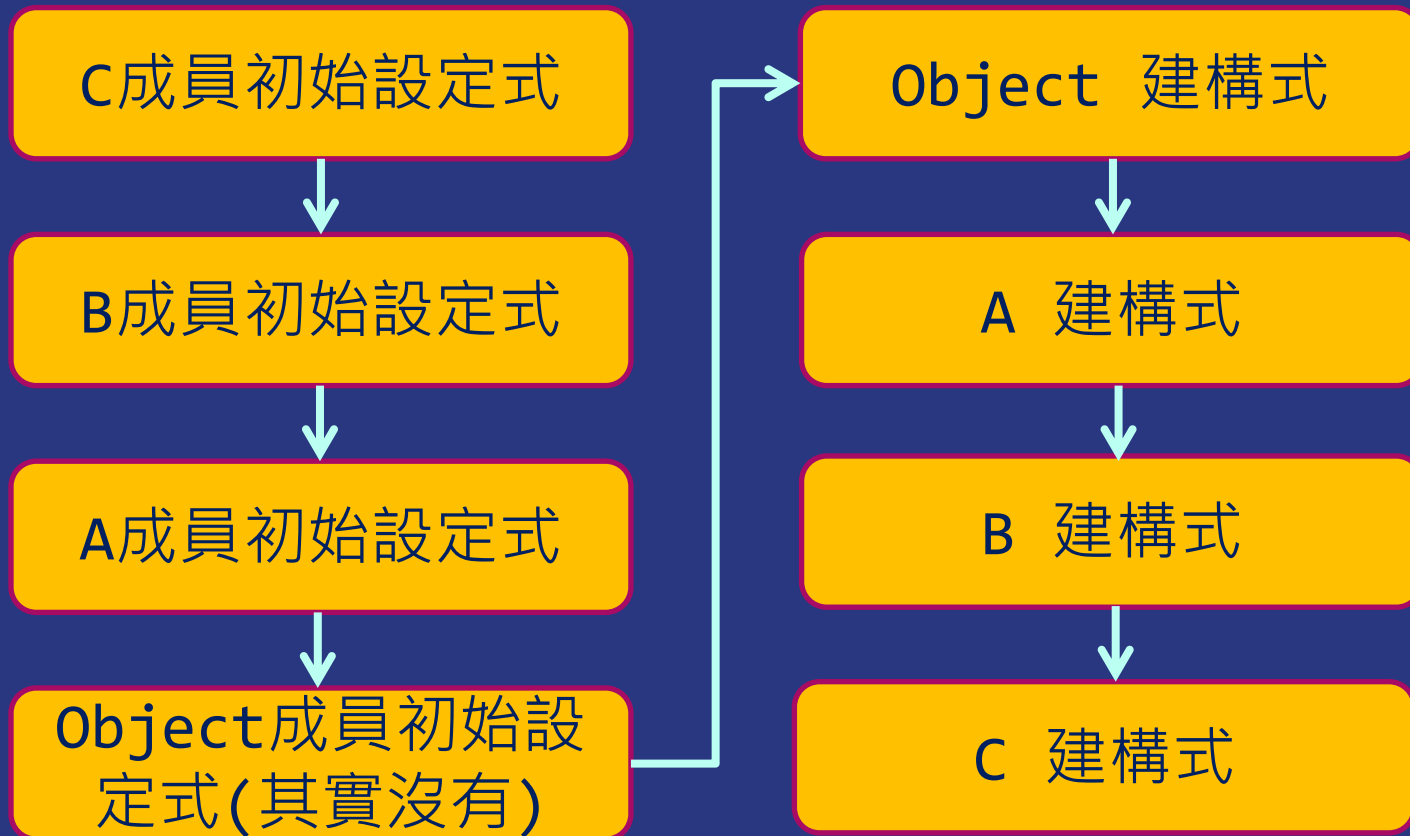
    public BaseClass(int y)
    {
        X = y;
    }
}
```



直接呼叫 BaseClass(int y) 並將 0 傳給它

在建構式中看到：**this** 表示呼叫自己的另一個建構式

繼承鍊：System.Object -- A -- B -- C



單元測試

測試簡述

- 軟體測試有很多種分類
 - 單元測試、整合測試...
 - 白箱測試、黑箱測試...
 - 等等.....
- 測試的目的
 - 確保軟體的運作與預期是相符的
- 現在要教的是單元測試

單元測試

- 通常單元測試是在測試單一行為的正確性
- 通常只針對 `public` 成員測試
- 想了解更多測試請看 [在 .NET 中測試](#)

C# 使用的單元測試框架

- MSTestv2
- Xunit
- NUnit

單元測試程式的特點 (MSTestv2)

- (1) 類別會帶有 `TestClass` attribute
- (2) 方法會帶有 `TestMethod` attribute
- (3) 最基本的測試結果驗證是使用 `Assert` 類別

```
[TestClass()]
public class RectangleOperationTests
{
    [TestMethod()]
    public void ComputeAreaTest()
    {
        Assert.Fail();
    }
}
```

Assert 類別

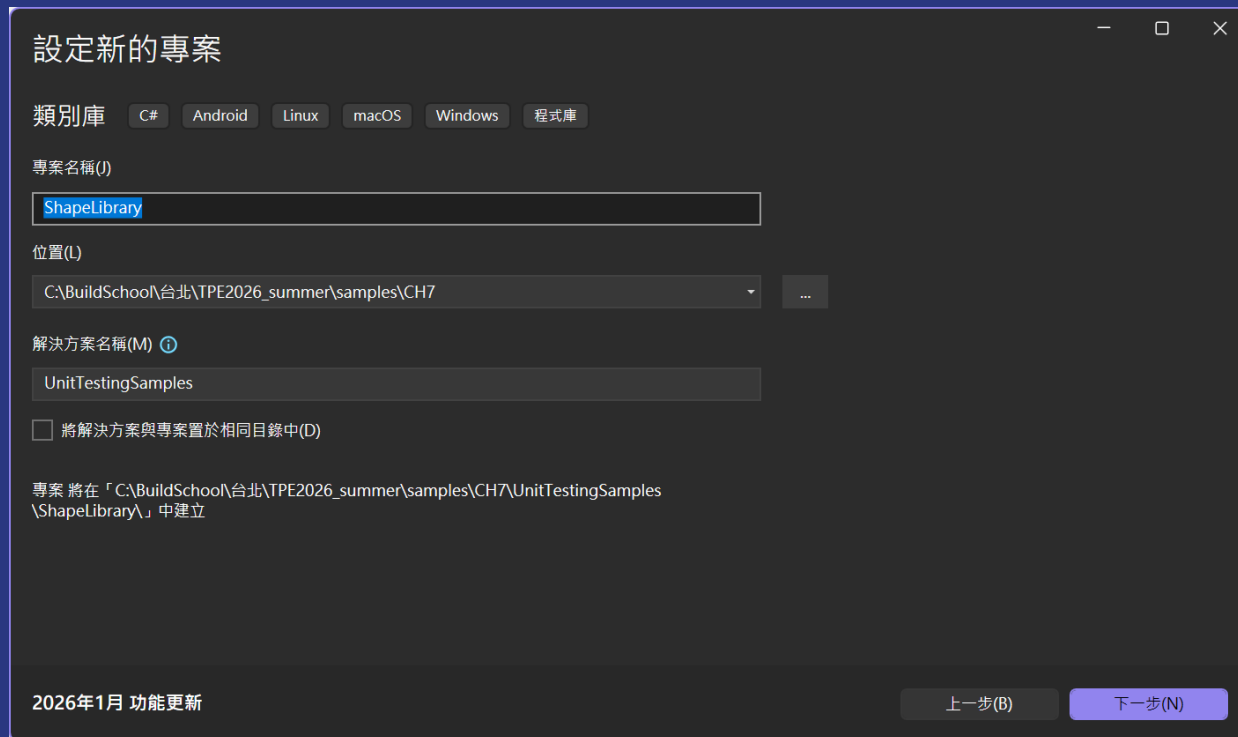
- Assert 包含許多相關於測試的方法
- 我們從 `Assert.AreEqual` 學起
- 基本的 `Assert.AreEqual` 方法會傳入兩個參數
 - 第一個參數為 `expected` 表示我們預期的結果
 - 第二個參數為 `actual` 表示被測試的程式所傳回的結果
 - 若 `expected` 和 `actual` 相等，則測試結果為成功
 - 若兩者不相等，則測試結果為失敗
 - 所以一般我們會在測試方法內用 `expected` 和 `actual` 當作相對應的變數名稱
- `Assert.AreEqual` 有泛型的版本

LAB

單元測試練習

加入新專案

- 方案名稱：UnitTestingSamples
- 專案名稱：ShapeLibrary
- 範本：類別庫 (class library)



在專案中建立一個 IShape interface

```
public interface IShape
{
    double GetArea();
    double GetPerimeter();
}
```

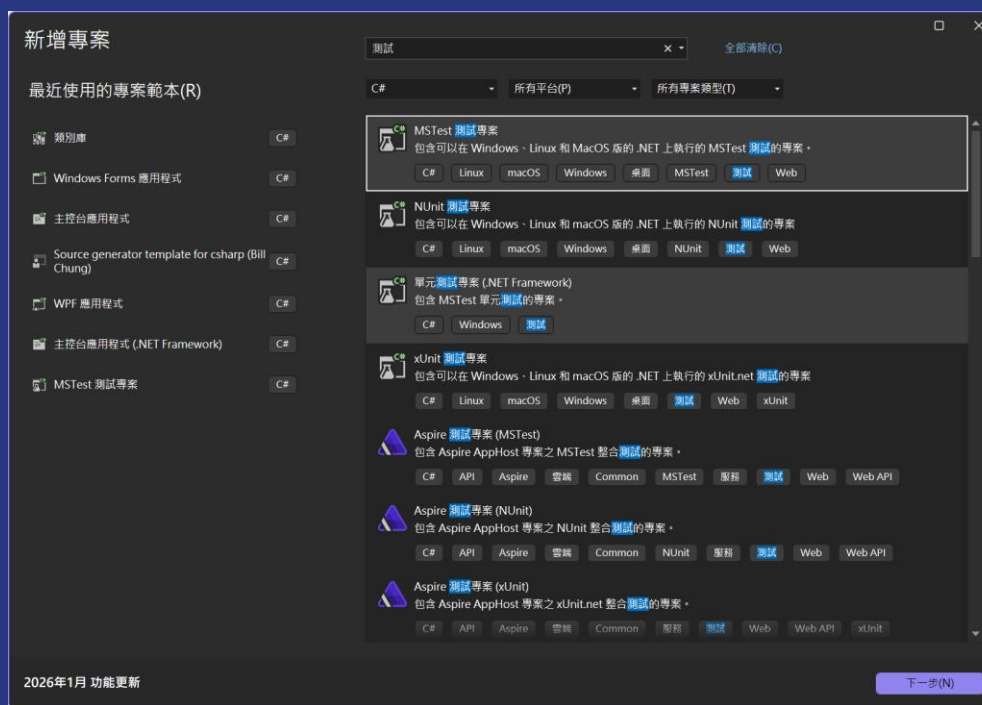
使用 IShape 介面實作兩個類別

```
public class MyRectangle : IShape
{
    public double Width { get; set; }
    public double Height { get; set; }
    public MyRectangle(double width, double height)
    {
        Width = width;
        Height = height;
    }
    public double GetArea() => Width * Height;
    public double GetPerimeter() => 2 * (Width + Height);
}

public class MyCircle : IShape
{
    public double Radius { get; set; }
    public MyCircle(double radius)
    {
        Radius = radius;
    }
    public double GetArea() => System.Math.PI * Radius * Radius;
    public double GetPerimeter() => 2 * System.Math.PI * Radius;
}
```

加入新專案

- 專案名稱：ShapeLibraryTest
- 範本：MSTest 測試專案



設定新的專案

MSTest 測試專案

C#

Linux

macOS

Windows

桌面

MSTest

測試

Web

專案名稱(I)

ShapeLibraryTest

位置(L)

C:\BuildSchool\台北\TPE2026_summer\samples\CH7\ShapeLibrary

專案 將在「C:\BuildSchool\台北\TPE2026_summer\samples\CH7\ShapeLibrary\ShapeLibraryTest
\'」中建立

2026年1月 功能更新

上一步(B)

下一步(N)

其他資訊

MSTest 測試專案

C#

Linux

macOS

Windows

桌面

MSTest

測試

Web

架構(F) ⓘ

.NET 10.0 (長期支援)

☐ 使用 MSTest.Sdk ⓘ

測試執行器 ⓘ

Microsoft.Testing.Platform

涵蓋範圍工具 ⓘ

Microsoft.CodeCoverage

固件 ⓘ

無

2026年1月 功能更新

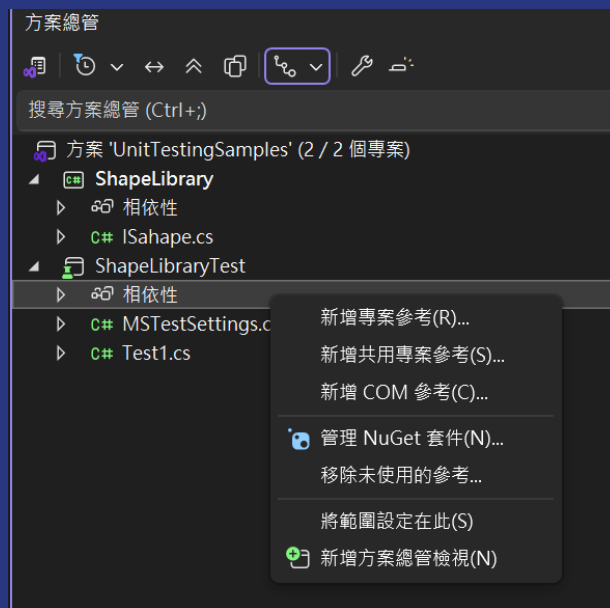
上一步(B)

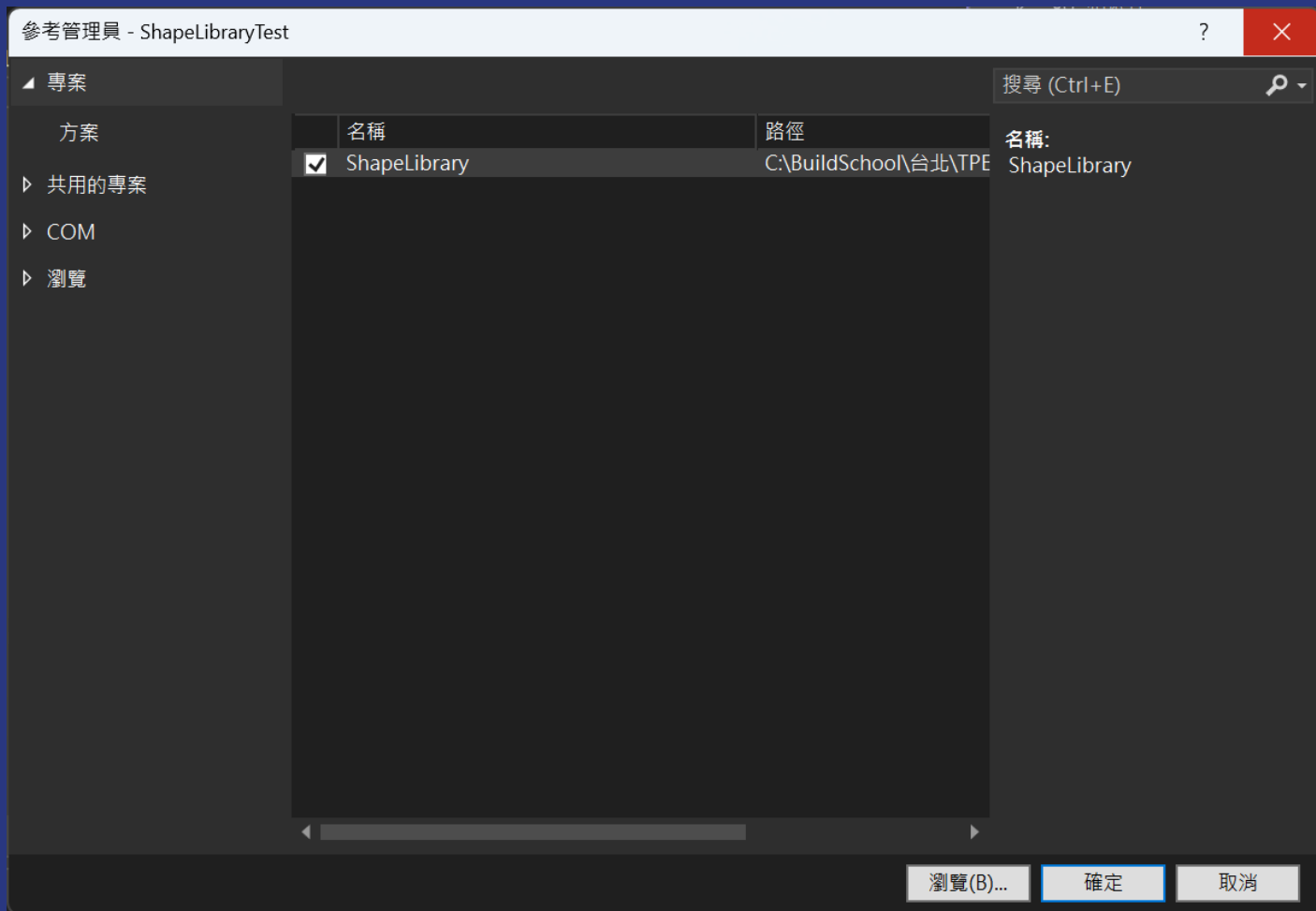
建立(C)

我們會看到 Visual Studio 建立以下的程式碼

```
namespace ShapeLibraryTest
{
    [TestClass]
    public sealed class Test1
    {
        [TestMethod]
        public void TestMethod1()
        {
        }
    }
}
```

加入測試對象相依性參考





單元測試方法的命名

- 有很多種命名的慣例，課程採用 **Given-When-Then (GWT)**
- **Given** 代表輸入
- **When** 代表行為（也就是你要測試的對象）
- **Then** 代表你預期的結果（也就是 **expected** [預期]）

(1) 將測試類別改名為MyRectangleTests

```
namespace ShapeLibraryTest
{
    [TestClass]
    public sealed class MyRectangleTests
    {
        [TestMethod]
        public void TestMethod1()
        {
        }
    }
}
```

Step 1

- (1) 修改 Test Method 的名稱
- (2) 這個名稱代表輸入一個 Rectangle，Width=6，Height=5；
執行 ComputArea 方法；預期得到的結果是 30

```
[TestMethod()]  
public void GivenRectangle_Width_6_Height_5_WhenComputeArea_Then30()  
{  
    Assert.Fail();  
}
```

Step 2

(1) 在 Test Method 中先加入預期值

```
[TestMethod()]  
public void GivenRectangle_Width_6_Height_5_WhenComputeArea_Then30()  
{  
    double expected = 30;  
}
```

Step 3

- (1) 在 Test Method 中加入要輸入的 MyRectangle
- (2) 設定 Rectangle 的 Width 與 Height 屬性值

```
[TestMethod()]  
public void GivenRectangle_Width_6_Height_5_WhenComputeArea_Then30()  
{  
    double expected = 30;  
    MyRectangle given = new MyRectangle(6, 5);  
}
```

Step 4

(1) 呼叫 GetArea 方法，並將結果值指派給 actual 變數 [這就是 when]

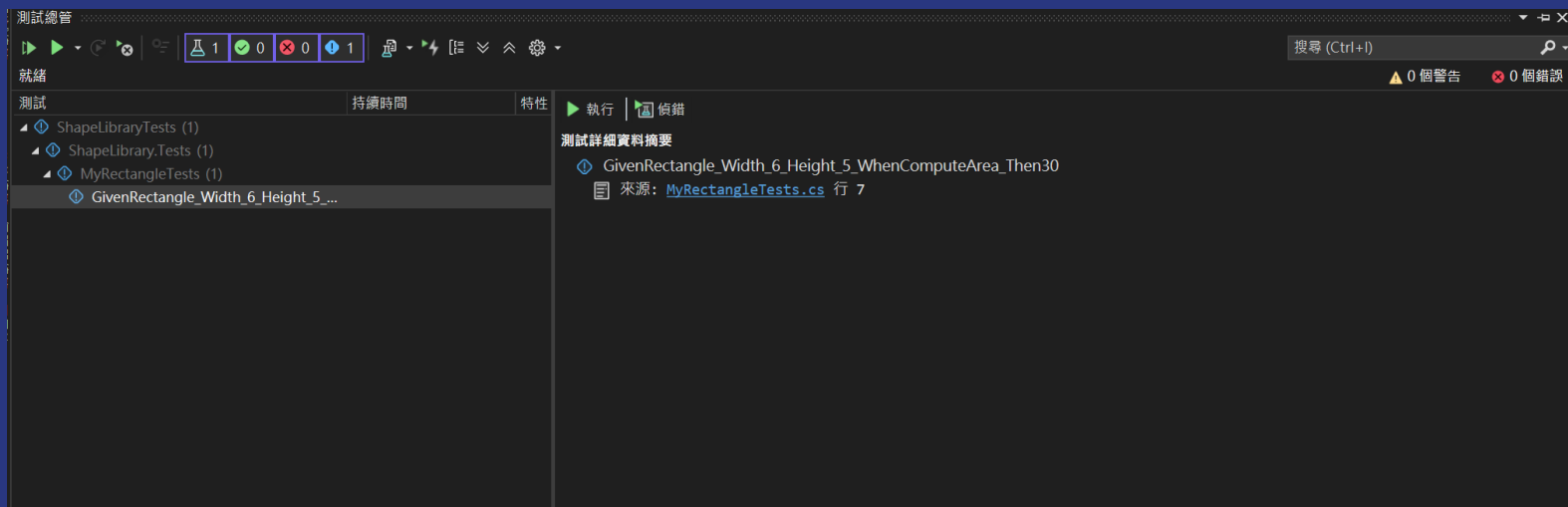
```
[TestMethod()]  
public void GivenRectangle_Width_6_Height_5_WhenComputeArea_Then30()  
{  
    double expected = 30;  
    MyRectangle given = new MyRectangle(6, 5);  
    double actual = given.GetArea();  
}
```

Step 5

- (1) 使用 `Assert.AreEqual` 比較 `expected` 與 `actual`
- (2) 請注意 `expected` 與 `actual` 的順序

```
[TestMethod()]
public void GivenRectangle_Width_6_Height_5_WhenComputeArea_Then30()
{
    double expected = 30;
    MyRectangle given = new MyRectangle(6, 5);
    double actual = given.GetArea();
    Assert.AreEqual(expected, actual);
}
```

Step 6 開啟測試總管，執行測試



LAB

多案例測試
(在原有的測試專案加入方法)

多案例測試

- 使用 `DynamicDataAttribute` 設定資料來源
 - 設定資料來源名稱
 - 設定資料來源為 `Method` 或 `Property` (我習慣用 `Method`)
 - 新版本可以自動偵測，不需要再設定這個選項

Step 1 建立測試方法並加入必要的 Attributes

```
[DynamicData(nameof(GetRectangleSource))]  
[TestMethod]  
public void Rectangle_GetAreaTest()  
{  
  
}
```

Step 2 建立資料來源的方法

- (1) 資料來源方法的宣告 `static IEnumerable<object[]>`
- (2) 資料內容會包含 `expected` (預期值)

```
private static IEnumerable<object[]> GetRectangleSource()
{
    // {width, height, expected}

    yield return new object[] { 6, 5, 30 };
    yield return new object[] { 7, 3, 21 };
    yield return new object[] { 9, 13, 117 };
}
```

Step 3 修改測試方法的參數列表

- (1) 參數一 **width**
- (2) 參數二 **height**
- (3) 參數三 **expected**

```
[DynamicData(nameof(GetRectangleSource))]  
[TestMethod]  
public void Rectangle_GetAreaTest(double width, double height, double expected)
```

Step 4 完成測試方法的內容

```
[DynamicData(nameof(GetRectangleSource))]  
[TestMethod]  
public void Rectangle_GetAreaTest(double width, double height, double expected)  
{  
    {  
        var given = new MyRectangle(width, height);  
        double actual = given.GetArea();  
        Assert.AreEqual(expected, actual);  
    }  
}
```

Step 5 開啟測試總管，執行測試

測試總管

測試探索已完成: 在 907.5 毫秒 找到 4 項測試

測試	持續時間	特性
▲ ✓ ShapeLibraryTest (4)	6 毫秒	
▲ ✓ ShapeLibraryTest (4)	6 毫秒	
▲ ✓ MyRectangleTests (4)	6 毫秒	
✓ GivenRectangle_Width_6_Height_5_WhenComput...	6 毫秒	
⚠ Rectangle_GetAreaTest(System.Double,System.Do...		

測試總管 工具箱 伺服器總管 SQL Server 物件總管

單元測試作業

- 說明

- 對你的黑客松累進稅率的程式碼進行單元測試
- 黑客松題目中有測試案例
- 需要全部案例都是綠燈

在這裏你將學到

Learn How to Learn

- 學新東西、新技術的能力
- 尋找解答的能力
- 隨時吸取新知識的能力

跟著你一輩子的能力 ...

對學生的好處

不只是就業銜接，培養自我解決問題的能力及信心!



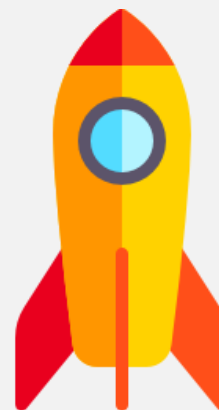
解決問題能力 (Problem-Solving)

- Learn how to learn
- Peer Learning
- Project-based



就業銜接 (Professional skills ready)

- Technical + Soft Skill



擴散及影響 (Make impacts)

- Mentor Networks